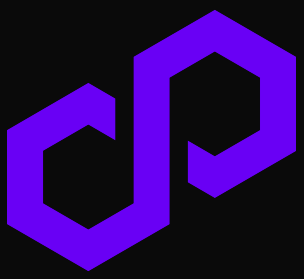
certora

Security Assessment



Polygon sPOL Staking

March 2026

Prepared for Polygon

Table of Contents

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Polygon sPOL Staking	https://github.com/0xPolygon/sPOL-contracts	eff5735 (first review) 4ef5382 (second review/fix review) 8add3b4 (third review and latest commit)	Solidity

Project Overview

This document describes the security review of **Polygon sPOL Staking**. The work was undertaken in 3 reviews:

First review:
Timeline - **December 1st, 2025** to **December 15th, 2025**.

The following contract list is included in our scope:

0xPolygon/sPOL-contracts/tree/dev/src/*
0xPolygon/pos-contracts/pull/37 (PR#37) at commit 9a19bb6.

Second review/fix review:
Timeline - **December 19th, 2025** to **December 30th, 2025**.

The following contract list is included in our scope:

0xPolygon/sPOL-contracts/tree/dev/src/*

Third review:
Timeline - **February 16th, 2026** to **February 23rd, 2026**.

The following contract list is included in our scope:

0xPolygon/sPOL-contracts/tree/dev/src/*

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

The sPOL protocol is a Liquid Staking Token (LST) system for the Polygon network, centered around the sPOL token which represents staked POL. Users deposit POL into the sPOLController on Layer 1 (Ethereum), where it is delegated to Polygon validators to earn staking rewards, while they receive sPOL tokens that appreciate in value as rewards accrue. The system includes a cross-chain component managed by sPOLMessenger and sPOLChild, enabling users on Layer 2 to mint and redeem sPOL seamlessly. The protocol manages liquidity through a "backfill" and "migration" mechanism, moving excess POL between layers to ensure withdrawals can always be processed. It also supports converting existing validator shares (dPOL) directly into sPOL, providing a unified liquid staking interface.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Security Overview

The sPOL protocol's safety depends on correct cross-chain state synchronization, the integrity of the L1-calculated exchange rate, and proper interaction with the Polygon StakeManager across multiple validator delegations. We focused on invariants around exchange rate monotonicity (the dPOL to sPOL ratio must strictly increase or stay flat), cross-chain token backing (L2 minted sPOL must be fully backed by L1 bridged sPOL or local surplus POL), L2 arbitrage constraints (effective exchange rates on L2 must account for yield lag via the safetyFee), and backfill state transitions (backfill cycles must strictly match between sPOLMessenger and sPOLChild). Attack vectors considered include cross-chain arbitrage exploiting stale L2 exchange rates, backfill deadlocks freezing L2 withdrawals and malicious validator accounting manipulation. Manual review covered all of the contracts in the /src directory.

Design Overview

The sPOLController (L1) acts as the source of truth, routing POL deposits into a curated list of Polygon validators based on target shares and a maxDivergence capacity limit. It handles all reward restaking, protocol fee minting (rewardFee), and unbonding execution. The sPOLChild (L2) acts as a high-speed storefront, processing instant POL/sPOL swaps using a cached exchange rate pushed from L1. To protect the protocol from arbitrage during the lag between L1 yield accrual and L2 rate syncs, sPOLChild applies a safetyFee on deposits and enforces a maxExchangeRateUpdateDelay circuit breaker.Because L2 liquidity can become imbalanced, the design introduces an asynchronous Backfill and Migration state machine. Surplus L2 POL is "migrated" to L1 for staking, while L2 sPOL burns trigger "backfills" where L1 unbonds stake and bridges POL back to L2. This creates significant configuration and operational complexity—particularly around the PolBridger, which is required to physically burn native POL on L2 (via MRC20) and submit Merkle proofs on L1 because native POL lacks a standard bridge withdrawFor function. Lastly, the design correctly isolates risk by delegating execution authority to an AccessManager, but relies heavily on off-chain keepers to push exchange rates and complete backfills, making liveness highly dependent on off-chain operational discipline.

Threat Model

Assets

• **Native POL and MATIC Tokens:** The underlying backing assets locked in the Polygon `StakeManager` or held in the L1/L2 protocol buffers.

• **sPOL Token:** The liquid staking derivative minted on L1 and L2 representing the user's fractional claim on the `dPOL` pool.

• **Protocol Fee Revenue:** The `rewardFee` (up to 10%) taken from the liquid rewards, and the `safetyFee` (up to 1%) used to compensate for the L2 exchange rate update lag.

• **Exchange Rate Data:** The `totaldPOLBalance / totalsPOLBalance` ratio dictated by L1 and cached on L2, dictating the minting/burning curves.

Actors

• **Protocol Admin / AccessManager:** Governance authority configuring fees, `maxDivergence`, validator whitelists, bridge addresses, and executing rescue functions.

• **sPOLController (L1):** Autonomous manager routing POL to `StakeManager` contract, calculating the canonical exchange rate and buying validator shares from their corresponding `ValidatorShare` contract.

• **sPOLChild (L2) & sPOLMessenger (L1):** Cross-chain coordinators handling local mints/burns, state sync messaging, and bridging.

• **Keepers / Permissionless Callers:** External actors triggering `balanceWithL1()`, `updateL2ExchangeRate()`, `completeBackfill()`, and reward restaking.

• **End Users:** Stakers and unstakers utilizing the protocol on either chain.

Trust Assumptions

• **Admin / Keeper Operational Discipline:** Proper tuning of the L2 `safetyFee` and `maxExchangeRateUpdateDelay` to outpace L1 yield volatility. Timely keeper execution of exchange rate updates and backfill completions; failure to update halts L2 trading completely.

Calling `restakeAllActiveValidators` often to account for the liquid rewards. (this is permissionless but it's better for a trusted entity to manage it as well)

• **Bridge Liveness and Integrity:** The Polygon PoS bridge accurately relays State Sync messages and MRC20 exit proofs without censorship, message corruption, or extended delays.

• **StakeManager Correctness:** The Polygon staking contracts (`StakeManager`, `ValidatorShare`) accurately report balances, correctly compound liquid rewards, and strictly enforce unbonding delays without accounting flaws.

Attack Vectors

• **Cross-Chain Slippage DoS:** Natural L1 exchange rate appreciation (surpassing the 0.3% L2 safety fee) while a migration message travels cross-chain breaks strict slippage checks upon L1 arrival, freezing the migration pipeline and deadlocking L2 state. [H-01]

• **Dead Token Accumulation on Local Settlement:** Asymmetric L2 buys and sells that are netted and settled locally fail to properly burn the differential sPOL, leaving orphaned token supply permanently trapped within the controller. [M-01]

• **Backfilled Liquidity Loss via Zero Address:** An uninitialized `child` state variable defaults to `address(0)`, causing the L1 `DepositManager` to bridge backfilled POL into an unrecoverable black hole on L2, leading to severe L2 insolvency. [C-02]

• **Total Supply Burn on Partial Backfill:** L2 `requestBackfill` incorrectly instructs L1 to burn the entire net circulating L2 supply debt (`bridgeMissingSPOL`) to satisfy a partial L2 withdrawal, completely destroying the cross-chain 1:1 token backing. [C-03]

• **Permissionless Backfill Deadlock:** `withdrawPOL` allows any user to process matured withdrawals on behalf of the L1 messenger prior to `completeBackfill` execution. This consumes the withdrawal nonces, forcing `completeBackfill` to revert and permanently bricking the L2 withdrawal pipeline. [C-04]

• **Exchange Rate Manipulation via Double-Counting:** `_buySPOLWithDPOLMulti` omits the local `totalStaked` update for incoming validator shares. Subsequent `reloadAllActiveValidatorInfo` executions double-count these deposited shares, permanently inflating the exchange rate and stealing value from future withdrawers. [C-05]

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	5	5	5
High	7	7	6
Medium	17	17	11
Low	4	4	3
Informational	3	3	-
Total	36	36	25

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
			Likelihood	

Detailed Findings

ID	Title	Severity	Status
C-01	Migrations from L2 to L1 can result in a severe skewing of the exchange rate under certain conditions, and eventually fund theft	Critical	Fixed
C-02	Uninitialized child address causes permanent loss of backfilled liquidity	Critical	Fixed
C-03	requestBackfill uses the wrong amount of sPOL to burn on L1	Critical	Fixed
C-04	Anyone is able to permanently DoS backfills by calling withdrawPOL on behalf of the L1 messenger	Critical	Fixed
C-05	Missing totalStaked update in _buySPOLWithDPOLMulti causes permanent exchange rate inflation and fund loss	Critical	Fixed
H-01	Migrations on L1 can be DOSd if L1/L2 exchange rate difference surpasses safety fee	High	Acknowledged
H-02	_buySPOLWithDPOLMulti fails to track restaked rewards in migration path, leading to exchange rate inflation	High	Fixed
H-03	_requestBackfill miscalculates request amounts, causing L1 revert due to exchange rate mismatch	High	Fixed

H-04	requestBackfill logic on L2 is wrongly calculating the mint/burn ratio leading to users being unable to sell SPOL	High	Fixed
H-05	sellSPOL functionality can be DOSd due to withdrawal claims in the same backfillLifecycle, as well as actualQuickRedeemReserve will be out-of-sync	High	Fixed
H-06	L2 → L1 migrations can be DOSd by donating DPOL tokens prior to a reload call	High	Fixed
H-07	Misinterpretation of ValidatorShare return values	High	Fixed
M-01	During local settlements, the difference between locally minted/toBeBurned always remains as "dead" sPOL in the contract	Medium	Acknowledged
M-02	There's an edge case in which locallyMinted == locallyToBeBurned, so no backfills/migrations can be requested, but at the same time withdraws can't be covered	Medium	Fixed
M-03	_maxDeposit doesn't account for the case in which totalSupply = 0	Medium	Acknowledged
M-04	After a validator unstakes, migrations as well as buying spol with dpol can be DoSD	Medium	Fixed
M-05	lastSuccessfulSellValidator is never set and used	Medium	Fixed

M-06	cleanUpMaticPOL fails to use contract balance and incorrectly pulls/transfer funds from/to recipient	Medium	Fixed
M-07	cleanUpMaticPOL reverts due to missing MATIC approval for migration	Medium	Fixed
M-08	exchangeRate updateDelay is too loose and sellSPOL doesn't check it, which can lead to users trading at an "outdated" exchange rate	Medium	Fixed
M-09	Missing slippage protection in buySPOL and sellSPOL	Medium	Acknowledged
M-10	Incorrect unit comparison in _sellSPOLSingle allows bypassing validator withdrawal limits	Medium	Fixed
M-11	_withdrawPOL incorrectly deletes withdrawal nonce 0, leading to fund loss	Medium	Fixed
M-12	Front-running of buySPOLPermit allows redirection of stake to arbitrary validators	Medium	Acknowledged
M-13	_applyPermit uses incorrect token contract, bricking buySPOLWithDPOLPermit	Medium	Fixed
M-14	Underflow in _migrateValidator caused by restaked amount discrepancy	Medium	Fixed
M-15	Front-running of permit signatures causes griefing in sPOLController	Medium	Acknowledged

M-16	DPOL and POL are used interchangeably which will lead to multiple issues	Medium	Acknowledged
M-17	lastSuccessfulBuyValidator and lastSuccessfulSellValidator are mistaken to be array indexes while they are validatorIds	Medium	Fixed
L-01	restakeAllActiveValidators reverts entirely if even a single validator is locked	Low	Acknowledged
L-02	Unchecked feeReceiver in initialization can permanently brick fee collection	Low	Fixed
L-03	Nested restricted modifier on takeFee breaks role separation in changeFeeReceiver	Low	Fixed
L-04	addValidator function performs no validations to make sure that the validator being added isn't an already existing one	Low	Fixed
I-01	_buySPOLWithDPOLMulti restakes after determining capacity which might lead to the overfunding of certain validators	Informational	Acknowledged
I-02	_sellVoucher_new should have internal visibility instead of public	Informational	Acknowledged
I-03	Insufficient precision for validator deposit shares limits stake distribution granularity	Informational	Acknowledged

Critical Severity Issues

C-01: Migrations from L2 to L1 can result in a severe skewing of the exchange rate under certain conditions, and eventually fund theft

Severity: Critical	Impact: High	Likelihood: High
Files: src/sPOLChild.sol	Status: Fixed	

Description:

Under certain conditions when migrating funds from L2 to L1, the wrong exchange rate will be reported, resulting in an over-backed sPOL which will skew the exchange rate and allow anyone to withdraw more sPOL tokens than they're supposed to.

```
function _requestMigration() internal {
        onGoingMigration = true;

        uint256 polToMigrate = actualQuickRedeemReserve -
targetQuickRedeemReserve;
        require(polToMigrate > 0, NothingToMigrate());

        actualQuickRedeemReserve -= polToMigrate;
        polBalance -= polToMigrate;

        locallyMintedSPOL -= locallyToBeBurnedSPOL;
        backMigratingSPOL = locallyMintedSPOL;
        locallyToBeBurnedSPOL = 0;
        locallyMintedSPOL = 0;

        _exitPOLforMessenger(polToMigrate);
        _sendMessageToRoot(
            abi.encode(MsgType.L2_MIGRATION_REQUEST,
_encodeL2MigrationRequestMessage(polToMigrate, backMigratingSPOL))
        );
        emit MigrationRequested(polToMigrate, backMigratingSPOL);
    }
```

• Imagine both locallyMinted and locallyToBeBurned are 0, and the POL/sPOL ratio is 1:1.

• Someone decides to sell 10_000e18 sPOL, so the locallyToBeBurned is 10_000e18, due to having no funds in the redeem reserve, this invokes the slow sell mechanism and marks the 10_000e18 as a missingWithdrawPOLBalance and puts it into a backfill cycle awaiting for backfill to be called.

• But prior to the operator invoking the backfilling mechanism through the balanceWithL1, someone goes on to buy 12_000e18 sPOL with 12_000e18 POL, so now locallyMinted is 12_000e18 (The 0.3% is omitted for easier accounting here, the issue persists with that as well).

• This also means that the quickRedeemReserve is 12_000e18 too. Now only the user can decide to withdraw prior to having the backfill initiated, i.e. they can decide not to do so through the quickRedeemReserve, and no one can withdraw on their behalf.

• From this point forward, if the operator calls the `balanceWithL1` this will invoke migrations as locallyMinted > locallyToBeBurned (12k vs 10k).

```
function _balanceWithL1() internal returns (bool) {
      require(!onGoingMigration, MigrationAlreadyOngoing());
      require(!onGoingBackfill, BackfillAlreadyOngoing());

      if (locallyToBeBurnedSPOL > locallyMintedSPOL) {
          _requestBackfill();
      } else if (locallyToBeBurnedSPOL < locallyMintedSPOL) {
          _requestMigration();
      } else {
          return false;
      }
      return true;
  }
```

• When migration is invoked, the `polToMigrate` = 12_000e18 (as the reserve is 12k), and `backMigratingSPOL` = 2_000e18 (The difference between minted and to be burned).

• Since the checks on L1 will bass as 12_000e18 will produce way more sPOL tokens than 2_000e18, we will mint 2_000e18 sPOL tokens while taking in 12_000e18 POL tokens ->

artificially skewing the exchange rate -> This will allow users to claim way more POL when selling their sPOL due to the "update".

Recommendations:

Pending withdrawals need to be taken into account as well when performing migrations so that the ratio stays consistent.

Customer Response:

Fixed in an architectural redesign.

Fix Review:

Confirmed.

C-02: Uninitialized child address causes permanent loss of backfilled liquidity

Severity: Critical	Impact: High	Likelihood: High
Files: src/sPOLMessenger.sol#L122	Status: Fixed	

Description:

The sPOLMessenger contract defines a state variable address public child; which is intended to store the address of the sPOLChild contract on the L2 (child) chain. However, this variable is never set in the constructor or the initialize function. It remains at its default value of address(0).

In the completeBackfill function, this zero address is passed to the depositManager as the recipient of the funds:

```
depositManager.depositERC20ForUser(address(polToken), child, totalWithdraw);
```

This triggers the following sequence of failure:

1. **L1 Action:** The `DepositManager` locks the POL tokens on L1 and sends a state sync message to L2 to mint/release the corresponding tokens to `child` (which is `address(0)`).

2. **L2 Action (Token Failure):** On L2, the bridge attempts to mint or transfer the POL tokens to `address(0)`. Polygon POL implementation reverts when minting or transferring to the zero address. This causes the state sync for the token transfer to fail on L2.

3. **L2 Action (Accounting Success):** Simultaneously, `completeBackfill` sends a separate `L1_BACKFILL_RESPONSE` message to the `sPOLChild` via the message tunnel (using `childTunnel`, which is set correctly). The `sPOLChild` receives this message, executes `_handleBackfillResponse`, and increments its `polBalance` as if the funds had arrived.

Impact:

• **Fund Loss:** The POL tokens are locked in the `DepositManager` on L1, but they are never credited to the `sPOLChild` contract on L2 because the mint/transfer to `address(0)` fails.

- **Insolvency:** The `sPOLChild` contract on L2 updates its accounting (`polBalance`), believing it has received the funds. Users attempting to withdraw against this phantom balance will fail due to insufficient actual liquidity in the contract.
- **Irreversible State:** Since `completeBackfill` marks the cycle as completed on L1 (`completedBackfill[_backFillCycle] = true`), the backfill cannot be retried with a correct address.

Recommendations:

The child variable must be properly initialized. Since childTunnel (from BaseRootTunnel) presumably holds the same address (the sPOLChild contract), the code should either use childTunnel directly or set child to equal childTunnel in the constructor/initializer.

Customer Response:

Fixed in commit 85dfdbd.

Fix Review:

Confirmed.

C-03: requestBackfill uses the wrong amount of sPOL to burn on L1

Severity: Critical	Impact: High	Likelihood: High
Files: src/sPOLChild.sol#L371	Status: Fixed	

Description:

The requestBackfill function is used to request liquidity (POL) from L1 to satisfy pending withdrawals on L2. When constructing the L1 request message, the function calculates two values:

1. `backFillAmount`: The amount of POL needed on L2 (e.g., 50 POL).
2. `bridgeMissingSPOL`: Calculated as `locallyMintedSPOL - locallyToBeBurnedSPOL`. This effectively represents the **entire net circulating supply** of sPOL on L2 that is backed by L1.

The critical flaw is that bridgeMissingSPOL sends the *total* L2 supply debt to L1, even if the backfill is only for a small portion of it.

When this message is processed on L1 (_handleBackfill -> startBackfillSell):

1. The `sPOLController` burns the full `bridgeMissingSPOL` amount (e.g., 100 sPOL).
2. It only unstakes/withdraws the `backFillAmount` (e.g., 50 POL).

Example:

Assume a POL/sPOL ratio of 1:1

1. UserA buys 100 sPOL by depositing 100 POL
2. UserA withdraws 50 of those tokens.
3. UserB buys 100 sPOL by depositing 100 POL
4. We migrate the POL balance which is 150 POL and we lock 150 sPOL inside the sPOLMessenger

5. UserA wants to withdraw the rest of his tokens - 50 so he calls the `sellSPOL` however we would need to backfill POL to complete this withdrawal as the sPOLChild doesn't have any (it's migrated)

6. When `requestBackfill` is called it calculates `bridgeMissingSPOL = 150 - 50 = 100` (burning will succeed here) so it will send a message with `backFillAmount = 50` and `bridgeMissingSPOL = 100` which on L1 would burn the 100 sPOL and withdraw only 50 POL.

7. We are now left with 50 sPOL on L1 and 100 sPOL on L2

This leads to insolvency as now the L2 sPOL is not backed by the expected amount on L1.

Recommendations:

The amounts in the backfill request must be matched correctly. The amount of sPOL burned on L1 must correspond exactly to the amount of POL being withdrawn, converted at the current exchange rate.

Customer Response:

Fixed in an architectural redesign.

Fix Review:

Confirmed.

C-04: Anyone is able to permanently DoS backfills by calling withdrawPOL on behalf of the L1 messenger

Severity: Critical	Impact: High	Likelihood: High
Files: src/sPOLMessenger.sol	Status: Fixed	

Description:

The process of "backfilling" after the message is sent from L2 to L1 is two-fold.

First the message will automatically trigger _handleBackfill :

```
 function _handleBackfill(bytes memory _msg) internal {
        (uint256 _polAmount, uint256 _sPOLAmount, uint256 _backFillCycle) =
_decodeL2BackfillRequestMessage(_msg);
        require(
            sPOLToken.balanceOf(address(this)) >= _sPOLAmount,
            NotEnoughSPOLInMessenger(_sPOLAmount,
sPOLToken.balanceOf(address(this)))
        );
        uint256[] memory nonces = sPOLController.startBackfillSell(_polAmount,
_sPOLAmount);
        backfillNonces[_backFillCycle] = nonces;
        emit BackfillStarted(_backFillCycle, _polAmount, _sPOLAmount);
    }
```

_handleBackfill will initiate startBackfillSell on the sPOLController which is supposed to execute the sell order and sell shares from validator(s) based on their withdraw availability:

```
function startBackfillSell(uint256 _amountPOL, uint256 _amountSPOL)
        external
        nonReentrant
        returns (uint256[] memory)
    {
        require(msg.sender == sPOLMessenger, AddressUnauthorized(msg.sender));
        uint256 expectedSPOL = convertPOLtoSPOL(_amountPOL);
```

```
        require(
            _amountSPOL >= expectedSPOL,
            BadExchangeRate(_amountPOL, _amountSPOL, totaldPOLBalance -
feedPOLBalance, totalsPOLBalance())
        );
        _takeSPOL(_amountSPOL, msg.sender);
        (uint16[] memory validator, uint256[] memory amount) =
_selectValidators(_amountPOL, false);
        uint256[] memory nonces = new uint256[](validator.length);
        for (uint256 i = 0; i < validator.length; i++) {
            uint256 userNonce =
_sellSharesFromValidator(validators[validator[i]], amount[i]);
            uint256 nonce =
                _addUserWithdrawNonceDetails(msg.sender, validator[i],
uint128(amount[i]), uint96(userNonce));
            nonces[i] = nonce;
        }
        emit sPOLBackfilled(msg.sender, _amountSPOL, _amountPOL);
        _emitExchangeRateUpdate();
        return nonces;
    }
```

Based on said availability and the number of validators which will handle the backfill, the withdrawal nonces are stored as part of the userNonces mapping, which will later be used to withdraw the POL amount once said withdrawals are finished after the withdrawal delay.

```
function _addUserWithdrawNonceDetails(address _user, uint16 _validatorId, uint128
_amount, uint96 _validatorNonce)
        internal
        returns (uint256)
    {
        uint256 nonce = globalWithdrawNonce++;
        userNonces[_user].push(nonce);
        NonceDetails storage details = withdrawNonceDetails[nonce];
        details.amount = _amount;
        details.validatorId = _validatorId;
        details.validatorNonce = _validatorNonce;
        return nonce;
```

```
    }
```

Said nonces are returned to the sPOLMessenger as part of the `_handleBackfill` function and will be stored in the backfillNonces mapping, the key which is used to store the nonces is based on the backfill nonce from the sPOLChild, which is incremented with each backfill.

It's intended that someone/anyone calls completeBackfill on the messenger after the POL is available in the controller, so that the backfill is finalized.

The actual problem is the following function on the controller, which allows anyone to initiate withdrawals on behalf of anyone:

```
function withdrawPOL(address _user, uint256 _nonce) external whenNotPaused {
        _withdrawPOL(_user, _nonce);
    }
```

Once this is initiated on behalf of the messenger address set as the _user argument, prior to completeBackfill being called:

```
function _withdrawPOL(address _user, uint256 _nonce) internal {
        uint256[] storage nonces = userNonces[_user];
        for (uint256 i = 0; i < nonces.length; i++) {
            if (nonces[i] == _nonce) {
                uint256 shares = _redeemNonceAtValidator(_nonce);
                require(shares > 0, WithdrawNotReady(_nonce));

                nonces[i] = nonces[nonces.length - 1];
                nonces.pop();

                polToken.transfer(_user, shares);
                emit POLWithdrawn(_user, shares, _nonce);
                return;
            }
        }
        revert NonceNotFound(_user, _nonce);
    }
```

The POL Tokens will be transferred to the messenger and the userNonce will be removed, this means that the POL tokens will be stuck in the messenger without a way to complete the backfill:

```
 function completeBackfill(uint256 _backFillCycle) external whenNotPaused
nonReentrant {
        require(!completedBackfill[_backFillCycle],
BackfillAlreadyCompleted(_backFillCycle));
        uint256 balanceBefore = polToken.balanceOf(address(this));
        for (uint256 i = 0; i < backfillNonces[_backFillCycle].length; i++) {
            sPOLController.withdrawPOL(backfillNonces[_backFillCycle][i]);
        }
        uint256 balanceAfter = polToken.balanceOf(address(this));
        uint256 totalWithdraw = balanceAfter - balanceBefore;
        depositManager.depositERC20ForUser(address(polToken), child,
totalWithdraw);
        _sendMessageToChild(
            abi.encode(MsgType.L1_BACKFILL_RESPONSE,
_encodeL1BackfillResponseMessage(totalWithdraw, _backFillCycle))
        );
        completedBackfill[_backFillCycle] = true;
        emit BackfillCompleted(_backFillCycle, totalWithdraw);
    }
```

Once the above function calls withdrawPOL, it will revert as the user nonces no longer exist as they were already withdrawn. This will permanently DoS the backfill flow, and the users won't be able to withdraw their POL on L2.

This attack vector can also be performed via withdrawPOL(address _user)as well.

Recommendations:

Either remove the functionality to finish POL withdrawals on behalf of someone else or modify the backfill functionality to take this scenario into account as well.

Customer Response:

Fixed in an architectural redesign.

Fix Review:

Confirmed.

C-05: Missing totalStaked update in _buySPOLWithDPOLMulti causes permanent exchange rate inflation and fund loss

Severity: Critical	Impact: High	Likelihood: High
Files: src/sPOLController.sol#L448	Status: Fixed	

Description:

The _buySPOLWithDPOLMulti function allows users to deposit existing Validator Shares (dPOL) in exchange for sPOL. While the function correctly updates the global totaldPOLBalance to reflect the incoming assets, it fails to update the totalStaked mapping for the specific validator (_validatorOfDPOL) from which the shares were transferred.

This desynchronization creates an accounting flaw that manifests when reloadAllActiveValidatorInfo is called.

reloadAllActiveValidatorInfo**:** This function reconciles the protocol's internal accounting with its actual balances:

```
ValidatorInfo storage validator = validators[activeValidators[i]];
amountToReduce += validator.totalStaked; // (1) Uses the STALE, incorrect value
validator.totalStaked = validator.validatorContract.balanceOf(address(this)); //
(2) Fetches the REAL, higher value
amountToAdd += validator.totalStaked; // (3) Adds the REAL value
// ...
totaldPOLBalance = totaldPOLBalance - amountToReduce + amountToAdd;
```

Because totalStaked was never incremented during the deposit, amountToReduce is smaller than it should be. amountToAdd, however, reflects the true balance (which includes the deposited shares). The result is that totaldPOLBalance is increased by the deposited amount a second time.

For example:

- Assume we have `ValidatorA.totalStaked = 200; ValidatorB.totalStaked = 0;` and we have 200sPOL. sPOL/POL = 1:1 (for simplification)

1. A user decides to buy SPOL with DPOL and does so with 100 DPOL shares from validatorA
    1. After the execution of `_buySPOLWithDPOLMulti` we would have the following state:
    2. totaldPOL = 300 (correct)
    3. validatorA.totalStaked = 100 (incorrect, should be 200)
    4. validatorB.totalStaked = 100 (correct)
2. Now when we call `reloadAllActiveValidatorInfo` we would have:
    1. After the first iteration of the loop we have `amountToReduce += 100` and `amountToAdd += 200` because none of the original validator shares were moved and the controller still holds 200 of his tokens
    2. After the second iteration of the loop we have `amountToReduce += 100` and `amountToAdd += 100`
    3. the final step is `totaldPOLBalance = 300 - 200 + 300 = 400` while the expected is 300 and the balance of the controller is 300
3. The sPOL /POL exchange is now impacted and instead of it being 1:1 it is now `300/400 = 0.75`
    1. The first user now redeems his sPOL for POL and he would be given `200*400/300 = 266.66(6)` POL tokens instead of 200 as expected.
    2. The second user can now only claim `300 - 266.66(6)` instead of 100, basically losing funds while the first withdrawer gains funds

Recommendations:

The _buySPOLWithDPOLMulti function must immediately update the totalStaked for the input validator to reflect the new deposit.

Customer Response:

Fixed in an architectural redesign.

Fix Review:

Confirmed.

High Severity Issues

H-01: Migrations on L1 can be DOSd if L1/L2 exchange rate difference surpasses safety fee

Severity: High	Impact: High	Likelihood: Medium
Files: src/sPOLMessenger.sol	Status: Acknowledged	

Description:

When buying sPOL on L2, the rate in which the sPOL is exchanged for POL is based on the L1 rate which is relayed cross-chain from L1 to L2, and is supposed to be updated frequently.

When utilizing that exchange rate to calculate the sPOL amount that needs to be minted, the current design of the sPOLChild system (the sPOL side on L2) includes a 0.3% fee when purchasing sPOL in order to mitigate the possibility of the L1 exchange rate being slightly stale and not accounting for the latest reward accrual:

```
    function convertPOLToSPOL(uint256 _polAmount) public view returns (uint256) {
        if (_polAmount == 0) {
            return 0;
        }
        return
            (_polAmount * l1SPOLBalance * (SAFETY_FEE_DENOMINATOR - safetyFee))
                / (l1DPOLBalance * SAFETY_FEE_DENOMINATOR);
    }
```

Whenever there is a surplus of POL (i.e. more buys than sells were executed since last re-balance), and furthermore balacneWithL1 is invoked, the POL surplus will be migrated to L1, and an adequate amount of sPOL will be minted based on that POL amount:

```
if (surplusPOL > 0) {
                if (locallyMintedSPOL >= locallyToBeBurnedSPOL) {
                    uint256 sPOLToBeMinted = locallyMintedSPOL -
locallyToBeBurnedSPOL;
```

```
                    locallyMintedSPOL = 0;
                    locallyToBeBurnedSPOL = 0;
                    _requestMigration(surplusPOL, sPOLToBeMinted);
function _requestMigration(uint256 _polToMigrate, uint256 _spolToMint) internal {
        onGoingMigration = true;
        backMigratingSPOL = _spolToMint;
        polBalance -= _polToMigrate;

        _exitPOLforMessenger(_polToMigrate);
        _sendMessageToRoot(
            abi.encode(MsgType.L2_MIGRATION_REQUEST,
_encodeL2MigrationRequestMessage(_polToMigrate, _spolToMint))
        );
        emit MigrationRequested(_polToMigrate, _spolToMint);
    }
```

When the message arrives on Root (i.e. L2), the _handleMigration logic will be invoked on the sPOLMessenger which is supposed to handle the L2 mint and subsequent sPOL locking, so that the amount minted on L2 is backed on L1:

```
function _handleMigration(bytes memory _msg) internal {
        (uint256 _polAmount, uint256 _mintedSPOL) =
_decodeL2MigrationRequestMessage(_msg);
        polBridger.takePOLL1(_polAmount);
        uint256 polBalance = polToken.balanceOf(address(this));
        require(polBalance >= _polAmount, NotEnoughPOLInMessenger(_polAmount,
polBalance));

        uint256 requiredPOL = sPOLController.convertSPOLtoPOL(_mintedSPOL) + 1;
        sPOLController.buySPOL(requiredPOL);
        // send surplus POL to controller, to be cleaned up later
        polToken.transfer(address(sPOLController), polBalance - requiredPOL);
        require(
            sPOLToken.balanceOf(address(this)) >= _mintedSPOL,
            NotEnoughSPOLInMessenger(_mintedSPOL,
sPOLToken.balanceOf(address(this)))
        );
```

The problem arises if the exchange rate on L1 has moved (increased) beyond a 0.3% difference in the meantime while the message was "traveling" to L1. As a simple example let's assume that the exchange ratio was 1:1 on both L1 and L2:

- User mints 997e18 sPOL in exchange for 1000e18 POL on L2;
- While that amount is being migrated to L1, the exchange rate moves beyond a 0.3% (*the different ways in which the exchange rate can move beyond 0.3% are explained below*);
- Assuming that now 1005e18 POL are needed to mint 997e18 sPOL, the `buySPOL` call will fail as not enough POL is present in the messenger.
- The contract will be put in a state of DoS which will not only affect the L1 side since the migration won't be able to be completed, but also L2 since the `onGoingMigration` flag will be true which means that exchange rate updates can be performed as well as no rebalances.

The exchange rate can appreciate beyond 0.3% in multiple ways, including, but not limited to:

- A reload call which will result in the validator stake being updated and it being more (DPOL can be directly donated to validators), and subsequently being accounted during the reload call.
- An admin calling `cleanUpMaticPOL`; POL is accumulated with every rebalancing as the 0.3% buffer is always sent to the controller in the form of POL, the similar is also true for extra sPOL being converted to POL and sent to the controller during backfills.

Customer Response:

This is an inherent property of the system, and we see no way to fix it. The plan is to run with the current approach (frequent syncs to avoid stale rate, and large enough safety fee that normal rewards won't pose a danger) and evaluate the usage of the L2 mechanism. If we deem the risk/reward relation as not worth it we will disable it and move to a bridge to L1 - stake/unstake - bridge to L2 approach.

H-02: _buySPOLWithDPOLMulti fails to track restaked rewards in migration path, leading to exchange rate inflation

Severity: High	Impact: Medium	Likelihood: High
Files: src/sPOLController.sol	Status: Fixed	

Description:

In the _buySPOLWithDPOLMulti function, the protocol handles deposits of existing Validator Shares (dPOL). The function has two paths:

1. **Direct Deposit (If block):** The shares stay with the input validator. The code correctly updates `totalStaked += restakedAmount + _amount`.
2. **Migration/Distribution (Else block):** The shares are distributed to other validators.

In the else block, the code calls restakeAndTransferFrom. This function compounds existing rewards (restakedAmount) and transfers the user's shares (_amount) to the controller.

The restakedAmount physically increases the controller's share balance in the _validatorOfDPOL contract. However, the code never adds restakedAmount to validators[_validatorOfDPOL].totalStaked.

Example:

2 active validators with 0 max dPOL each and current totaldPOL = 100. validator1.totalStaked = 50 and validator2.totalStaked = 50

1. A user buys sPOL by depositing 100 dPOL of validator1
2. restakedAmount = 5 for validator1 so `totaldPOL = 105`. `validator1.totalStaked` stays at 50
3. Now in the for loop of selected validators restakedAmount of validator2 = 5 and we update `validator2.totalStaked = 55` and `totaldPOL = 110`.

4. `validator2.totalStaked += 50 = 105`
5. `validator1.totalStaked += 50 = 100` (incorrect, should be 105)
6. `dPOlBalance = 210`
7. Now reload and dPOLBalance becomes 215 instead of the expected 210.

Recommendations:

In the else block of _buySPOLWithDPOLMulti, ensure the restakedAmount is added to the input validator's totalStaked.

Customer Response:

Fixed in an architectural redesign.

Fix Review:

Confirmed.

H-03: _requestBackfill miscalculates request amounts, causing L1 revert due to exchange rate mismatch

Severity: High	Impact: Medium	Likelihood: High
Files: src/sPOLChild.sol	Status: Fixed	

Description:

The _requestBackfill function manages the synchronization of withdrawals between L2 and L1. It calculates two key values for the cross-chain message:

1. `backFillAmount`**:** derived from `pendingWithdrawPOLBalance` (plus any reserve top-up). This represents the total POL needed to satisfy user withdrawals on L2.

2. `sPOLToSell`**:** calculated as `locallyToBeBurnedSPOL - locallyMintedSPOL`. This represents the net sPOL liability that L2 is sending back to L1.

The vulnerability arises because the function nets out the sPOL liability (locallyMintedSPOL) but fails to net out the corresponding POL assets (backFillAmount).

Scenario:

Assume 1:1 ratio for sPOL:POL

1. User A buys 50 sPOL on L2

2. 50 POL is migrated to L1

3. User B buys 25 sPOL

4. User A wants to withdraw his 50 POL so he does that which would do a slow sell and `missingWithdrawPOLBalance = 50`

5. Now we balanceL1 and since `locallytoBeBurned > locallyMinted` we do a backfill where `pendingWithdrawPOLBalance = 50` and so `backFillAmount = 50` but when it goes to

calculate the `sPOLToSell` it will be 50 - 25 = 25 so it will try burn 25 sPOL and get 50 POL on L1 which would fail as the ratio is far off.

The backfill process is broken (DoS) whenever locallyMintedSPOL > 0 and locallyToBeBurnedSPOL > locallyMintedSPOL. Users cannot finalize withdrawals requiring L1 liquidity in this state.

Recommendations:

The _requestBackfill logic must utilize the local POL liquidity that corresponds to the netted locallyMintedSPOL.

Customer Response:

Fixed in an architectural redesign.

Fix Review:

Confirmed.

H-04: requestBackfill logic on L2 is wrongly calculating the mint/burn ratio leading to users being unable to sell SPOL

Severity: High	Impact: Medium	Likelihood: High
Files: src/sPOLChild.sol	Status: Fixed	

Description:

In the sPOLChild, whenever a sell can't be covered by the current POL balance of the contract, a.k.a. actualQuickRedeemReserve, it enters a "slow sell mode" in which the withdrawal request is "stored" and a backfill process is initiated in order to get POL from L1.

The problem is that the current backfill process tries to transfer and burn the difference between the locally minted and "burned" SPOL which is wrong, and will in most cases lead to reverts and a DOS of the backfill logic.

Here's the root cause of the issue within the requestBackfill function:

```
if (locallyMintedSPOL > locallyToBeBurnedSPOL) {
            bridgeMissingSPOL = locallyMintedSPOL - locallyToBeBurnedSPOL;
            _burnSPOLForMessenger(bridgeMissingSPOL);
            locallyMintedSPOL -= locallyToBeBurnedSPOL;
            locallyToBeBurnedSPOL = 0;
        }
```

Let's take the following hypothetical situation as an example with step-by-step contract state changes:

- Users exchange 50_000e18 POL to SPOL:
  - 50_000e18 * 500_000e18 * (10_000 - 50) / (520_000e18 * 10_000) = 47_836.5e18
  - `locallyMintedSPOL` = 47_836.5e18;
  - `actualQuickRedeemReserve` = 50_000e18;
  - `polBalance` = 50_000e18;

- A `requestMigration` call empties out the current reserve / polBalance:
  - `polBalance` = 0;
  - `actualQuickRedeemReserve` = 0;
- User(s) decide to sell 10_000e18 SPOL
  - `locallyToBeBurnedSPOL` = 10_000e18;
  - `polToReturn` = (10_000e18 * 520_000e18) / 500_000e18 = 10_400e18;
- Considering that the contract doesn't have any pol balance and the `actualQuickRedeemReserve` is 0, this will enter "slow sell SPOL mode" and the withdrawal will get registered in the next backFillCycle.
- `requestBackfill` is called in order to fulfill this withdrawal:

```
if (locallyMintedSPOL > locallyToBeBurnedSPOL) {
            bridgeMissingSPOL = locallyMintedSPOL - locallyToBeBurnedSPOL;
            _burnSPOLForMessenger(bridgeMissingSPOL);
            locallyMintedSPOL -= locallyToBeBurnedSPOL;
            locallyToBeBurnedSPOL = 0;
        }
```

- Considering that `locallyMintedSPOL` is greater than `locallyToBeBurnedSPOL` (47,836.5e18 vs. 10_00e18), the code within the if clause will get executed.
  - `bridgeMissingSPOL` = 37,836.5e18
  - `_burnSPOLForMessenger(37_836.5e18)`:
- We will try to transfer and burn 37_836.5e18 SPOL tokens, while the contract only holds 10_000e18 SPOL tokens. This will effectively DOS the backfill system.

Recommendations:

The SPOL amount transferred and burned should be the SPOL amount which corresponds to the polToReturn amount, or in the above case 10_000e18 SPOL.

Customer Response:

Fixed in an architectural redesign.

Fix Review:

Confirmed.

H-05: sellSPOL functionality can be DOSd due to withdrawal claims in the same backfillLifecycle, as well as actualQuickRedeemReserve will be out-of-sync

Severity: High	Impact: High	Likelihood: Medium
Files: src/sPOLChild.sol	Status: Fixed	

Description:

In the sPOLChild, whenever a sell can't be covered by the current POL balance of the contract, a.k.a. actualQuickRedeemReserve it enters a "slow sell mode" in which the withdrawal request is "stored" and a backfill process is initiated in order to get POL from L1.

The problem is that users awaiting for their POL to be backfilled for a withdraw can prematurely claim it in the same backfill cycle in which it's supposed to arrive if POL becomes available in the contract because of buys, without updating the actualQuickRedeemReserve which will cause it to be out of sync and will lead to the sellSPOL functionality being DOSd.

Here's an example:

- A user sells 10_000e18 SPOL tokens in exchange for 10_400 POL tokens.

```
function _slowSellSPOL(uint256 _polAmount) internal {
        missingWithdrawPOLBalance += _polAmount;
        UserOutstanding memory userOutstanding =
            UserOutstanding({outstandingPOL: _polAmount, backFillCycle:
backFillCycle + 1, nonce: globalWithdrawNonce});
        userOutstandingPOL[msg.sender].push(userOutstanding);
    }
```

The current contract state:

- `outstandingPOL`: 10_400e18
- `backFillCycle`: x(**current**) + 1;

It should also be kept in mind that bridging can take up to an hour;

After the outstanding backfill request was created, requestBackfill can be invoked in order to initiate the process:

```
function requestBackfill() external whenNotPaused nonReentrant {
        require(!onGoingBackfill, BackfillAlreadyOngoing());
        onGoingBackfill = true;
        backFillCycle += 1;
        pendingWithdrawPOLBalance += missingWithdrawPOLBalance;
        uint256 backFillAmount = pendingWithdrawPOLBalance;

        if (actualQuickRedeemReserve < targetQuickRedeemReserve) {
            backFillAmount += targetQuickRedeemReserve -
actualQuickRedeemReserve;
        }
        missingWithdrawPOLBalance = 0;

      ...
    }
```

- The important thing to note is that the `backFillCycle` is increased by 1, immediately which makes it X+1 (or the same cycle in which the user above can/should claim their tokens).

Now if we take a look at the logic below in the withdrawPOL:

```
 function withdrawPOL() external whenNotPaused nonReentrant {
        UserOutstanding[] storage outstandings = userOutstandingPOL[msg.sender];
        uint256 totalToWithdraw = 0;
        bool reordered; // false
        for (uint256 i = 0; i < outstandings.length; i++) {
            if (reordered) {
                reordered = false;
                i--;
            }
            if (completedBackfills[outstandings[i].backFillCycle]) {
                reservedWithdrawPOLBalance -= outstandings[i].outstandingPOL;
            } else if (outstandings[i].backFillCycle == backFillCycle) { //If
currentBackFillCycle equals the one in the outstandings, amount is decreased from
pending
```

```
                pendingWithdrawPOLBalance -= outstandings[i].outstandingPOL;
            } else if (outstandings[i].outstandingPOL <=
actualQuickRedeemReserve) { // IF outstandingPOL is less than
actualQuickRedeemReserve
                actualQuickRedeemReserve -= outstandings[i].outstandingPOL; //
it's decreased from quickRedeemReserve
                missingWithdrawPOLBalance -= outstandings[i].outstandingPOL;
//it's decreased from missing WithdrawPOLBalance
            } else { // otherwise we skip
                continue;
            }
            totalToWithdraw += outstandings[i].outstandingPOL; //totalToWithdraw
is increased by the amount
            emit POLWithdrawn(msg.sender, outstandings[i].outstandingPOL,
outstandings[i].nonce);

            outstandings[i] = outstandings[outstandings.length - 1]; //the
current outstanding is put at the last place
            outstandings.pop(); //last place is popped
            reordered = true; //reordered set to true
        }
        require(totalToWithdraw > 0, POLAmountMustBeGreaterThanZero());
        polBalance -= totalToWithdraw;
        (bool success,) = payable(msg.sender).call{value: totalToWithdraw}("");
        require(success, POLTransferFailed());
```

The following else if statement which is supposed to handle the case in which the current backfill cycle is the same as the on in which the user is supposed to claim their withdrawal, is actually where the problem arises:

```
else if (outstandings[i].backFillCycle == backFillCycle) { //If
currentBackFillCycle equals the one in the outstandings, amount is decreased from
pending
                pendingWithdrawPOLBalance -= outstandings[i].outstandingPOL;
```

Since this is true, if in the meantime someone bought SPOL and there's enough POL balance to cover the withdrawal, i.e. actualQuickRedeemReserve can cover it, the else if clause above will get

executed, but the problem is that pendingWithdrawPOLBalancewill be decreased, while at the same time actualQuickRedeemReserve remains the same.

So now whenever users try to sell SPOL:

```
function sellSPOL(uint256 _sPOLAmount) external whenNotPaused nonReentrant {
        _transfer(msg.sender, address(this), _sPOLAmount);
        locallyToBeBurnedSPOL += _sPOLAmount;
        uint256 polToReturn = convertSPOLToPOL(_sPOLAmount);
        emit sPOLBurned(msg.sender, _sPOLAmount, polToReturn,
++globalWithdrawNonce);
        if (actualQuickRedeemReserve >= polToReturn) {
            _quickSellSPOL(polToReturn);
        } else {
            _slowSellSPOL(polToReturn);
        }
    }
```

_quickSellSPOL will be called as actualQuickRedeemReserve is higher than it's supposed to be.

```
function _quickSellSPOL(uint256 _polAmount) internal {
        actualQuickRedeemReserve -= _polAmount;
        polBalance -= _polAmount;
        (bool success,) = payable(msg.sender).call{value: _polAmount}("");
        require(success, POLTransferFailed());
        emit POLWithdrawn(msg.sender, _polAmount, globalWithdrawNonce);
    }
```

- But `_quickSellSPOL` fails due to not enough balance in the contract.

An additional problem is that when the backfill-in-question arrives due to the pendingWithdrawPOLBalance, the "if-clause" will be true, and the actualQuickRedeemReserve will be increased even further:

```
 function _handleBackfillResponse(bytes memory _msg) internal {
        (uint256 _returnedPOL, uint256 _backFillCycle) =
_decodeL1BackfillResponseMessage(_msg);
        if (_returnedPOL > pendingWithdrawPOLBalance) {
```

```
            uint256 leftOver = _returnedPOL - pendingWithdrawPOLBalance;
            actualQuickRedeemReserve += leftOver;
            reservedWithdrawPOLBalance += pendingWithdrawPOLBalance;
            pendingWithdrawPOLBalance = 0;
        } else {
            pendingWithdrawPOLBalance -= _returnedPOL;
            reservedWithdrawPOLBalance += _returnedPOL;
        }
        polBalance += _returnedPOL;
        completedBackfills[_backFillCycle] = true;
        onGoingBackfill = false;
        emit BackfillCompleted(_returnedPOL, _backFillCycle);
    }
```

Recommendations:

Don't allow users to withdraw POL if the backfill hasn't been completed and/or handle the case properly by also decreasing the actualQuickRedeemReserve as well.

Customer Response:

Fixed in PR#4.

Fix Review:

Confirmed.

H-06: L2 → L1 migrations can be DOSd by donating DPOL tokens prior to a reload call

Severity: High	Impact: High	Likelihood: Medium
Files: src/sPOLController.sol	Status: Fixed	

Description:

When funds are bridged from L2 to L1 with a completeMigration call, these calls are permission-less, i.e. they can be executed by anyone through the sPOLChild contract.

There are two root causes to this issue, and making the DOS possible and executable either intentionally by a malicious entity or unintentionally due to given circumstances.

The first root cause is the "strict" check on L1 when the funds arrive, i.e. requiring the expectedSPOL to always be greater than or equal to the _amountSPOL.

```
  function completeMigration(uint256 _amountPOL, uint256 _amountSPOL) external
nonReentrant {
        require(msg.sender == sPOLMessenger, AddressUnauthorized(msg.sender));
        uint256 expectedSPOL = convertPOLtoSPOL(_amountPOL);
        require(
            expectedSPOL >= _amountSPOL,
            BadExchangeRate(_amountPOL, _amountSPOL, totaldPOLBalance -
feedPOLBalance, totalsPOLBalance())
        );
```

The second one is that reload calls are going to affect the total DPOL balance based on the current balance of the contract:

```
   function reloadAllActiveValidatorInfo() external restricted {
        uint256 amountToReduce;
        uint256 amountToAdd;
        for (uint256 i = 0; i < activeValidators.length; i++) {
            ValidatorInfo storage validator = validators[activeValidators[i]];
            amountToReduce += validator.totalStaked;
```

```
            validator.totalStaked =
validator.validatorContract.balanceOf(address(this));
            amountToAdd += validator.totalStaked;
        }
        totaldPOLBalance = totaldPOLBalance - amountToReduce + amountToAdd;
        _emitExchangeRateUpdate();
    }
```

While funds are en route to L1, someone could donate funds to the sPOLController contract prior to a reload call, which will affect the exchange rate due to the totaldPOLBalance change, and effectively DOS the migration functionality, as it will continue reverting due to the exchange rate mismatch.

Below is a practical hypothetical example which shows how this can be achieved:

- L2 sPOLChid:
  - Users want to exchange 50_000 POL to SPOL, the exchange rate is based on a totalSupply of 500_000e18 SPOL and 520_000e18 DPOL.
  - 50_000e18 * 500_000e18 * (10_000 - 50) / (520_000e18 * 10_000) = 47_836.5e18
  - locallyMintedSPOL = 47_836.5e18;
  - `actualQuickRedeemReserve` = 50_000e18;
  - `polBalance` = 50_000e18;
- Migration is requested:
  - sPOLToAddToBridge = 47_836.5e18
  - `polToMigrate` (50_000e18) is sent to bridge;
- When this arrives on L1 it will check whether the current rate results in more SPOL than what we sent from L2, and the 47_836.5e18 serves as slippage.
- What happens if while the funds are "traveling" as bridge time takes about an hour, there's a reload call and someone "donates" 20_000e18 dPOL to the sPOLController contract?
  - TotalSpol: 500_000e18
  - TotalDPOL: 540_000e18 (20k DPOL donation)

- 50_000e18 * 500_000e18 / 540_000e18 = 46_296e18 (This is achievable with a smaller donation as well).

- Call reverts because 46_296e18 isn't greater than or equal to 47_836e18.

Recommendations:

Reload calls should be performed only if migrations/backfills are locked on L2, and should result in an immediate exchange rate update sent from L1 to L2 after it.

Customer Response:

Fixed in an architectural redesign.

Fix Review:

Confirmed.

H-07: Misinterpretation of ValidatorShare return values

Severity: High	Impact: Medium	Likelihood: High
Files: src/sPOLController.sol#L457	Status: Fixed	

Description:

The sPOLController interacts with the Polygon ValidatorShare contract to stake POL. Specifically, it calls restakeAndStakePOL, which compounds existing rewards (liquidReward) and deposits new POL (_amount) in a single transaction.restakeAndStakePOL returns (amountToDeposit, liquidReward), where amountToDeposit is the sum of the fresh deposit plus the restaked rewards:

```
// ValidatorShare.sol
uint256 amountPlusReward = _amount.add(liquidReward);
uint256 amountToDeposit = _buyShares(amountPlusReward, ...);
return (amountToDeposit, liquidReward);
```

However, the sPOLController and more specifically _buySharesFromValidator incorrectly assumes amountDeposited is equal only to the fresh deposit _amount. This leads to three critical failures:

1. **DoS via Internal Check:** Inside `_buySharesFromValidator`, the code enforces: `require(amountDeposited == _amount, ...);` If the contract has *any* accrued rewards (which happens almost immediately after the first stake), `amountDeposited` (Principal + Rewards) will be greater than `_amount` (Principal), causing the transaction to revert.

2. **DoS via Upstream Check:** Even if the internal check is removed, `_buySharesFromValidator` returns `amountDeposited`. The calling function `_buySPOLMulti` aggregates these returns and enforces:
`require(totalShares == _amount, BuySharesMismatch(_amount, totalShares));`
This check will also fail because `totalShares` will include the restaked rewards, while `_amount` is only the user's input amount.

3. **Double Counting of Balance:** The accounting logic in `_buySharesFromValidator` adds the rewards to the total balance twice:

3. It also adds the rewards twice for the `validator.totalStaked`.
This permanently inflates `totaldPOLBalance` relative to the actual underlying assets, breaking the exchange rate calculation `convertSPOLtoPOL`.

Recommendations:

Update _buySharesFromValidator to correctly handle the compound return value. The function should recognize that amountDeposited includes the rewards and adjust the checks and accounting accordingly.

Customer Response:

Fixed in commit 8141747.

Fix Review:

Confirmed

Medium Severity Issues

M-01: During local settlements, the difference between locally minted/toBeBurned always remains as "dead" sPOL in the contract

Severity: Medium	Impact: Low	Likelihood: High
Files: src/sPOLChild.sol	Status: Acknowledged	

Description:

Whenever balanceWithL1 is invoked there are a few paths which can be taken in order to rebalance L2.

- The outstanding withdraws can be balanced and covered locally by POL already present in the contract if there's enough to cover them, they can be partially covered by said POL, and the rest or the whole amount can be requested from L1 via backfill;
- Any surplus POL is migrated to L1 via the migration path.

Whenever there's a local settlement involved, the sPOL tokens which should be burned remain as "dead" tokens within the sPOLController contract.

```
    function _balanceWithL1() internal {
        require(!onGoingMigration, MigrationAlreadyOngoing());
        require(!onGoingBackfill, BackfillAlreadyOngoing());


        uint256 surplusPOL = polBalance - reservedWithdrawPOLBalance;
        // more surplus than missing withdraw balance -> full local backfill
possible
        if (surplusPOL >= missingWithdrawPOLBalance) {
            if (missingWithdrawPOLBalance > 0) {
                _startBackfill(missingWithdrawPOLBalance);
                surplusPOL -= missingWithdrawPOLBalance;
```

```
                _localBackfill(missingWithdrawPOLBalance);
                _completeBackfill(0, backFillCycle);
            }
            if (surplusPOL > 0) {
                if (locallyMintedSPOL >= locallyToBeBurnedSPOL) {
                    uint256 sPOLToBeMinted = locallyMintedSPOL -
locallyToBeBurnedSPOL;
                    locallyMintedSPOL = 0;
                    locallyToBeBurnedSPOL = 0;
                    _requestMigration(surplusPOL, sPOLToBeMinted);
                } else {
                    // special case where the safety fee surplus is also
expressed as extra sPOL to be burned
                    emit BalancedOnlyLocally();
                }
            } else {
                // surplus matched exactly the missing withdraw balance, or both
were zero
                emit BalancedOnlyLocally();
            }
        } else {
            // not enough surplus, so request backfill
            _startBackfill(missingWithdrawPOLBalance);
            if (surplusPOL > 0) {
                _localBackfill(surplusPOL);
            }
            uint256 sPOLToBeBurned = locallyToBeBurnedSPOL - locallyMintedSPOL;
            locallyToBeBurnedSPOL = 0;
            locallyMintedSPOL = 0;
            _requestBackfill(missingWithdrawPOLBalance, sPOLToBeBurned);
            missingWithdrawPOLBalance = 0;
        }
    }
```

For example, if a user bought 1000e18 tokens, then another user sold 500e18:

- The 500e18 sPOL tokens which were sold, won't be burned, even though they were settled locally. The remainder, which is 500e18 POL will be sent to L1, and based on it 500e18 sPOL tokens will be minted and locked. (This is assuming a 1:1 ratio for easier accounting).

In the opposite scenario if a user bought 500e18 tokens, and then another user sold 1000e18 tokens:

- The difference of 500e18 sPOL tokens will be burned, but the remainder 500e18 sPOL tokens will be left in the contract.

In the first case a user mints 1000e18 sPOL tokens on L2, if we were to migrate them immediately, i.e. there's no pending withdraws, so no need to perform any local settlements, we would bridge the whole POL tokens and mint 1000e18 sPOL tokens on L1 and "lock them".

If we were to assume no prior supply on both L1 and L2 and a 1:1 ratio (just so it's easier to show the example):

- This would result in a totalSupply of 1000e18 sPOL tokens on L2. After migrating the 1000e18 POL from L2 to L1, we would then mint 1000e18 sPOL on L1, which would also result in a totalSupply of 1000e18 sPOL tokens on L1.

If that same user minted 1000e18 sPOL tokens on L2, and then sold 500e18 of them, we would have:

- A totalSupply of 1000e18 sPOL on L2 (as tokens were never burned), and after we "migrate" the 500e18 POL to L1, we would have a totalSupply sPOL of 500e18 locked on L1.
- A similar scenario occurs when sells are more than buys.

This is to show the discourse between when migrations/backfills are executed in a cross-chain matter, versus the ones which are settled locally.

Customer Response:

There would be 4 ways to avoid it and currently we don't want to use any as their downsides are larger than odd totalSupply, which some token on pos already exhibit due to the same limitations.

M-02: There's an edge case in which locallyMinted == locallyToBeBurned, so no backfills/migrations can be requested, but at the same time withdraws can't be covered

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/sPOLChild.sol	Status: Fixed	

Description:

balanceWithL1only accounts for cases in which burned < minted, and vice versa, but the case in which burned == minted isn't accounted for. There's an edge case in which if an exchange rate resulted in an update of more than 0.3%, pending withdraws can't be covered, while at the same time, no migrations/backfills can be requested.

```
function _balanceWithL1() internal returns (bool) {
      require(!onGoingMigration, MigrationAlreadyOngoing());
      require(!onGoingBackfill, BackfillAlreadyOngoing());

      if (locallyToBeBurnedSPOL > locallyMintedSPOL) {
          _requestBackfill();
      } else if (locallyToBeBurnedSPOL < locallyMintedSPOL) {
          _requestMigration();
      } else {
          return false;
      }
      return true;
  }
```

• Imagine the following scenario:

• Bob wants to exchange 1_000e18 POL for sPOL on L2, for simplicity let's assume that both the totalSupply and the totalDPOL are 10_000e18

• `(1000e18 * 10_000e18 * (10_000 - 30)) / (10_000e18 * 10_000) = 997e18;`

    - This will result in 997e18 sPOL, meaning that locallyMinted == 997e18, and the quickRedeemReserve == 1_000e18 POL.

- Let's assume that a reload call took place on L1 and this triggered an exchange rate update of above 0.3%, for the purpose of the example, let's say it's 0.35%, this will result in a `totalSupply == 10_997e18` and `totalDPOLBalance == 11_035e18`.

- A user wants to sell their 997e18 sPOL for POL:

    - `(997e18 * 11_035e18) / 10_997e18 = 1000.04e18` -> this will result in the user not being able to quick sell due to the redeem reserve having only 1_000e18 tokens, while the user needs 1000.04e18 as shown in the case above.

    - This will store a slow sell backfill request.

- But due to locallyMinted == locallyToBeBurned (997e18 == 997e18) - backfills can't be triggered.

Recommendations:

Account for this edge case scenario by either making sure that all backfills are cleared out prior to sending an exchange rate update, or artificially supply the difference by minting more sPOL.

Customer Response:

Fixed in an architectural redesign.

Fix Review:

Confirmed.

M-03: _maxDeposit doesn't account for the case in which totalSupply = 0

Severity: Medium	Impact: Low	Likelihood: Medium
Files: src/sPOLController.sol	Status: Acknowledged	

Description:

_maxDeposit never accounts for the case in which there's 0 totalStaked and 0 dPOL, i.e. prior to the first buy.

For example as it can be seen in _validatorWithHighestTotalStakeDistance where this case is properly handled, if 0 is returned during buys, it will return type(uint256).max.

```
if (_positive && maxDistance == 0) {
            maxDistance = type(uint256).max;
        }
```

Due to a lack of such a mechanism in _maxDeposit, this DoSs _buySPOLSingle on the first deposit.

_buySPOLMulti will be able to handle this as an exception as _selectValidators will distribute it equally among everyone.

If we take a look at _maxDeposit, myactualMaxShare will return a 0, meaning that users won't be able to buy SPOL through a single validator:

- `myBigShare`= 20 + 10 = 30 (examples of a depositShare and divergence);
- `theOtherShare`= 0;
- `restShare` = 100 - 30 = 70;
- `myactualMaxShare`= (0 * 30) / 70 = 0;

```
 function _maxDeposit(ValidatorInfo storage _validator) internal view returns
(uint256) {
        uint8 myBigShare = _validator.depositShare + maxDivergence;
        if (myBigShare >= 100) {
```

```
            return type(uint256).max;
        }
        uint256 theOtherShare = totaldPOLBalance - _validator.totalStaked;
        uint8 restShare = 100 - myBigShare;
        uint256 myactualMaxShare = (theOtherShare * myBigShare) / restShare;

        if (myactualMaxShare <= _validator.totalStaked) {
            return 0;
        }
        return myactualMaxShare - _validator.totalStaked;


    }
```

Recommendations:

Include a special if case in which if the totalSupply of SPOL is 0, to return type(uint256).max;

Customer Response:

Acknowledged.

M-04: After a validator unstakes, migrations as well as buying spol with dpol can be DoSD

Severity: Medium	Impact: High	Likelihood: Low
Files: src/sPOLController.sol	Status: Fixed	

Description:

The current design of the StakeManager, ValidatorShare and the sPOLController contracts permit users to sell their DPOL in exchange for POL when a validator begins the process of unstaking or after they've unstaked, but forbids them from buying new shares.

In the StakeManager, when _unstake is invoked, as it can be seen below, the deactivationEpoch is set as the next epoch and the following changes occur:

```
function _unstakeValidator(uint256 validatorId, bool pol) internal {
        Status status = validators[validatorId].status;
        require(
            validators[validatorId].activationEpoch > 0 &&
validators[validatorId].deactivationEpoch == 0
                && (status == Status.Active || status == Status.Locked)
        );

        uint256 exitEpoch = currentEpoch.add(1); // notice period
        _unstake(validatorId, exitEpoch, pol);
    }
```

The total delegated amount is then updated in the timeline, so when next epoch comes, the stake is removed and sent to the validator.

One crucial thing that occurs here is that the validatorShare contract is locked, so if someone tries to buy new shares, they can't (onlyWhenUnlocked modifier) will be invoked.

```
  function _buyShares(
        uint256 _amount,
        uint256 _minSharesToMint,
```

```
        address user
    ) private onlyWhenUnlocked returns (uint256) {
```

So when a user decides to sell their vouchers (whether through sPOLController or directly on the validator):

```
function _sellVoucher(
        uint256 claimAmount,
        uint256 maximumSharesToBurn,
        bool pol
    ) private returns (uint256, uint256) {
        // first get how much staked in total and compare to target unstake
amount
        (uint256 totalStaked, uint256 rate) = getTotalStake(msg.sender);
        require(totalStaked != 0 && totalStaked >= claimAmount, "Too much
requested");

        // convert requested amount back to shares
        uint256 precision = _getRatePrecision();
        uint256 shares = claimAmount.mul(precision).div(rate);
        require(shares <= maximumSharesToBurn, "too much slippage");

        _withdrawAndTransferReward(msg.sender, pol);

        _burn(msg.sender, shares);
        stakeManager.updateValidatorState(validatorId, -int256(claimAmount));
        activeAmount = activeAmount.sub(claimAmount);

        uint256 _withdrawPoolShare =
claimAmount.mul(precision).div(withdrawExchangeRate());
        withdrawPool = withdrawPool.add(claimAmount);
        withdrawShares = withdrawShares.add(_withdrawPoolShare);

        return (shares, _withdrawPoolShare);
    }
```

This is still possible, the shares will be burned and then the equivalent POL amount can later be withdrawn.

The issue arises within restakeAndTransferFrom. If buySPOLWithDPOL is called, it will later invoke restakeAndTransferFrom.

```
 IValidatorShare validatorOfDPOL =
IValidatorShare(stakeManager.getValidatorContract(_validatorOfDPOL));
        (bool success, uint256 restakedAmount) =
validatorOfDPOL.restakeAndTransferFrom(_user, address(this), _amount);
        require(success, DPOLRestakeTransferFromFailed());


 function restakeAndTransferFrom(address from, address to, uint256 value) public
returns (bool, uint256) {
        // Sender gets their rewards paid out (code from
_withdrawAndTransferReward)
        uint256 liquidRewardFrom = _calcAndResetReward(from);
        if (liquidRewardFrom != 0) {
            require(stakeManager.transferFundsPOL(validatorId, liquidRewardFrom,
from), "Insufficent rewards");
            stakingLogger.logDelegatorClaimRewards(validatorId, from,
liquidRewardFrom);
        }
        uint256 amountRestaked;
        // recipient already has POL staked with this validator? restake their
rewards
        if (balanceOf(to) > 0) {
            // rewardPerShare was updated when calling _calcAndResetReward above
            uint256 liquidRewardsTo = _calculateReward(to, rewardPerShare);
            if (liquidRewardsTo != 0) {
                // reusing liquidReward saves us a call here
                // restake rewards to reset initialRewardPerShare value (code
from _restake)
                amountRestaked = _buyShares(liquidRewardsTo, 0, to);

                if (liquidRewardsTo > amountRestaked) {
                    // return change to the user
                    _payout(liquidRewardsTo - amountRestaked, to, "Insufficent
```

```
rewards", true);
                    stakingLogger.logDelegatorClaimRewards(validatorId, to,
liquidRewardsTo - amountRestaked);
                }

                (uint256 totalStaked,) = getTotalStake(to);
                stakingLogger.logDelegatorRestaked(validatorId, to, totalStaked);
            }
        }
        // set "to" baseline to current to prevent claiming historical rewards
        // Do this after calling _calcAndResetReward to use the updated
rewardPerShare
        initalRewardPerShare[to] = rewardPerShare;

        // Call parent's transferFrom which checks allowance and transfers shares
        bool success = super.transferFrom(from, to, value);
```

The issue arises when the reward is calculated. The _calcAndResetReward calculation is for the from user, so technically it can be larger than the one saved for the to user which is the sPOLController:

```
  function _calcAndResetReward(address user) private returns (uint256) {
        uint256 _rewardPerShare =

_calculateRewardPerShareWithRewards(stakeManager.withdrawDelegatorsReward(validat
orId));
        uint256 liquidRewards = _calculateReward(user, _rewardPerShare);

        rewardPerShare = _rewardPerShare;
        initalRewardPerShare[user] = _rewardPerShare;
        return liquidRewards;
    }
```

This is also supplemented by the fact that the validator will also be eligible for a reward in the epoch between the _unstake call and the exitEpoch finishes.

So considering that the balanceOf the sPOLController can be larger than 0 (this can also be caused by a direct transfer to the contract), they can be eligible for rewards, so technically when _buyShares is invoked, this will of course revert due to the ValidatorShare contract being locked.

Manual migrations invoked by the admin team can also be DOSd as they require a mandatory restake (below is the _restake function on ValidatorShare):

```
function _restake(address user, bool pol) private returns (uint256, uint256) {
        uint256 liquidReward = _calcAndResetReward(user);
        uint256 amountRestaked;

        if (liquidReward != 0) {
            amountRestaked = _buyShares(liquidReward, 0, user);

            if (liquidReward > amountRestaked) {
                // return change to the user
                _payout(liquidReward - amountRestaked, user, "Insufficent
rewards", pol);
                stakingLogger.logDelegatorClaimRewards(validatorId, user,
liquidReward - amountRestaked);
            }

            (uint256 totalStaked,) = getTotalStake(user);
            stakingLogger.logDelegatorRestaked(validatorId, user, totalStaked);
        }

        return (amountRestaked, liquidReward);
    }
```

Now if there's a pending reward (again also mentioning the 1 epoch between _unstaking and exiting) this can revert. It can be mitigated by the validator calling unstakeClaim which will clear out the rewards, but since this can only be called by the validator - they can intentionally decide to DoS migrations and the sPOLController partially.

```
function _unstakeClaim(uint256 validatorId, bool pol) internal {
        uint256 deactivationEpoch = validators[validatorId].deactivationEpoch;
        // can only claim stake back after WITHDRAWAL_DELAY
        require(
```

```
            deactivationEpoch > 0 && deactivationEpoch.add(WITHDRAWAL_DELAY) <=
currentEpoch
                && validators[validatorId].status != Status.Unstaked
        );

        uint256 amount = validators[validatorId].amount;
        uint256 newTotalStaked = totalStaked.sub(amount);
        totalStaked = newTotalStaked;

        // claim last checkpoint reward if it was signed by validator
        _liquidateRewards(validatorId, msg.sender, pol);

        NFTContract.burn(validatorId);

        validators[validatorId].amount = 0;
        validators[validatorId].jailTime = 0;
        validators[validatorId].signer = address(0);

        signerToValidator[validators[validatorId].signer] =
INCORRECT_VALIDATOR_ID;
        validators[validatorId].status = Status.Unstaked;

        _transferToken(msg.sender, amount, pol);
        logger.logUnstaked(msg.sender, validatorId, amount, newTotalStaked);
    }
```

Recommendations:

Check if the validator has initiated their exit/unstake and if they have, don't re-stake rewards or accumulate them virtually.

Customer Response:

Fixed in an architectural redesign.

Fix Review:

Confirmed.

M-05: lastSuccessfulSellValidator is never set and used

Severity: Medium	Impact: Low	Likelihood: High
Files: src/sPOLController.sol	Status: Fixed	

Description:

The main purpose of lastSuccessfulBuyValidator and lastSuccessfulSellValidator is to intelligently distribute the stake to the next validator in line, or remove stake from someone else besides the last validator.

In all buy functions, we can see instances of the lastSuccessfulBuyValidator being set to the ID of the validator to which all the stake was distributed in single-type buy functions, or the last one who received the stake in multi-type buy functions:

```
 for (uint256 i = 0; i < amount.length; i++) {
            totalShares += _buySharesFromValidator(validators[validator[i]],
amount[i]);
        }
        lastSuccessfulBuyValidator = validator[validator.length - 1];
```

The problem is that in all sell functions, including _sellSharesFromValidator lastSucessfulSellValidator isn't set anywhere. This means that lastSucessfulSellValidator is basically redundant. In _selectValidators the lastSucessfulBuyValidator is used to determine the next validator in-line both during buys and sells:

```
for (uint256 i = 0; i < activeValidators.length; i++) {
            ValidatorInfo storage validator =
                validators[activeValidators[(lastSuccessfulBuyValidator + i) %
activeValidators.length]];

            uint256 maxAmount = _validatorMaxTotalStakeDistance(validator, _buy);
```

This means that the last buy validator will be used to determine the next validator during sells as well.

Recommendations:

Set the lastSuccessfulSellValidator in all instances when a sell of SPOL occurs, similar to how lastSuccessfulBuyValidator is set. Then lastSuccessfulSellValidator needs to be used to determine the next validator in-line for sales, rather than the lastSuccessfulBuyValidator one as it's currently being done.

Customer Response:

Fixed in PR#4.

Fix Review:

Confirmed. The lastSuccessfulBuyValidator and lastSuccessfulSellValidator functionality was removed.

M-06: cleanUpMaticPOL fails to use contract balance and incorrectly pulls/transfer funds from/to recipient

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/sPOLController.sol#L945	Status: Fixed	

Description:

The cleanUpMaticPOL function is intended to "clean up" any loose POL tokens sitting in the sPOLController contract by converting them to sPOL and sending them to a receiver.

The logic flow is:

1. Check contract balance: `uint256 polBalance = polToken.balanceOf(address(this));`
2. If `receiver != address(0)`, call `buySPOL(polBalance, _validator)`.
3. Transfer the resulting sPOL from the contract to the receiver: `sPOLToken.transfer(_receiver, mintedSPOL)`.

The buySPOL function (and its internal helper _buySPOLSingle) executes _takePOL(_amount, msg.sender). This pulls the POL tokens from the caller of the transaction (the admin), not from the contract itself (address(this)).

Consequences:

1. **Wrong Source of Funds:** The function attempts to charge the admin for the cleanup, rather than using the idle funds already in the contract. If the admin hasn't approved the contract to spend their personal POL, the transaction reverts.
2. **Wrong Destination of Mint:** `buySPOL` mints the sPOL to `msg.sender` (the admin).
3. **Transfer Failure:** The function then attempts `sPOLToken.transfer(_receiver, mintedSPOL)`. This tries to send sPOL from the *contract* to the receiver. Since the sPOL was minted to the *admin*, the contract has no sPOL to send, and the transaction reverts due to insufficient balance.

Recommendations:

Call buySPOL as an external call - this.buySPOL and give the required allowance to self.

Customer Response:

Fixed in PR#4.

Fix Review:

Confirmed. The receiver logic in cleanUpMaticPOL is removed and now it correctly funds a validator of choice.

M-07: cleanUpMaticPOL reverts due to missing MATIC approval for migration

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/sPOLController.sol#L936	Status: Fixed	

Description:

The cleanUpMaticPOL function is designed to convert any MATIC tokens held by the sPOLController into POL tokens using the polygonMigration contract. The execution flow is as follows:

1. The contract checks its MATIC balance.
2. If `maticBalance > 0`, it calls `polygonMigration.migrate(maticBalance)`.
3. The `polygonMigration` contract executes `legacy.safeTransferFrom(msg.sender, address(this), amount)`, attempting to pull the MATIC tokens from the `sPOLController`.

However, the sPOLController never approves the polygonMigration contract to spend its MATIC tokens. Neither the initialize function nor the cleanUpMaticPOL function calls maticToken.approve().That means if the contract has some MATIC balance the cleanUpMaticPOL will always fail.

Recommendations:

The cleanUpMaticPOL function must approve the polygonMigration contract to spend the MATIC tokens before calling migrate.

Customer Response:

Fixed in PR#4.

Fix Review:

Confirmed.

M-08: exchangeRate updateDelay is too loose and sellSPOL doesn't check it, which can lead to users trading at an "outdated" exchange rate

Severity: Medium	Impact: Medium	Likelihood: Low
Files: src/sPOLChild.sol	Status: Fixed	

Description:

In the sPOLChild contract, there's a maxExchangeRateUpdateDelay variable which is used to make sure that the difference between the last exchange update and the current block.timestamp isn't "too large".

```
maxExchangeRateUpdateDelay = 30 days;
// we get about 0,25% rewards in a month, so if we pause after a month of no
update
// 0,3% should be safe so sPOL doesn't become cheaper than L1
safetyFee = 30; // 0.3%
```

The first part of the issue is that the assumption mentioned above is wrong. The "0.25%" rewards per month isn't the only way in which the exchange rate can be moved. Users can theoretically move it by donating DPOL tokens directly to the L1 controller prior to a reload call, as the the new totalDPOLBalance (which is crucial during exchange rate calculations) will be based on the balance of the contract.

```
function reloadAllActiveValidatorInfo() external restricted {
        uint256 amountToReduce;
        uint256 amountToAdd;
        for (uint256 i = 0; i < activeValidators.length; i++) {
            ValidatorInfo storage validator = validators[activeValidators[i]];
            amountToReduce += validator.totalStaked;
            validator.totalStaked =
validator.validatorContract.balanceOf(address(this));
            amountToAdd += validator.totalStaked;
        }
        totaldPOLBalance = totaldPOLBalance - amountToReduce + amountToAdd;
```

```
        _emitExchangeRateUpdate();
    }
```

Theoretically, this means that the exchange rate can move more than the 0.25% per month which could lead to an “outdated“ exchange rate and a possible arbitrage opportunity between L1 and L2.

The second part of the issue is that sellSPOL lacks a maxExchangeRateUpdateDelay check, same as the one present in buySPOL:

```
  function buySPOL(uint256 _polAmount) external payable whenNotPaused {
        require(
            lastExchangeRateUpdate + maxExchangeRateUpdateDelay >=
block.timestamp,
            ExchangeRateUpdateTooOld(lastExchangeRateUpdate,
maxExchangeRateUpdateDelay, block.timestamp)
        );
```

sellSPOL:

```
 function sellSPOL(uint256 _sPOLAmount) external whenNotPaused nonReentrant {
        _transfer(msg.sender, address(this), _sPOLAmount);
        locallyToBeBurnedSPOL += _sPOLAmount;
        uint256 polToReturn = convertSPOLToPOL(_sPOLAmount);
        emit sPOLBurned(msg.sender, _sPOLAmount, polToReturn,
++globalWithdrawNonce);
        if (actualQuickRedeemReserve >= polToReturn) {
            _quickSellSPOL(polToReturn);
        } else {
            _slowSellSPOL(polToReturn);
        }
    }
```

Considering that the amount of POL which a user should receive is dependent on the L1 SPOL total supply and totalDPOLBalance - the same as during buys, it’s logical that exchange rate freshness is checked here as well, as during outdated exchange rates users would receive less POL than they’re entitled to at the time of sell.

Recommendations:

Include the maxExchangeRateUpdateDelay in the sell function as well, also either tighten the updateDelay to make sure that the exchange rate is “fresh“ or atomically trigger an exchange rate update after a reload call on L1.

Customer Response:

Fixed in an architectural redesign.

Fix Review:

Confirmed.

M-09: Missing slippage protection in buySPOL and sellSPOL

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/sPOLController.sol src/sPOLChild.sol	Status: Acknowledged	

Description:

The buySPOL (and related permit variants) and sellSPOL functions execute trades at the current exchange rate calculated by convertPOLtoSPOL and convertSPOLtoPOL. These functions take only an input amount (_amount) and do not allow the user to specify a minimum output amount (minOut).

The exchange rate between sPOL and POL is dynamic and changes whenever:

1. Rewards are restaked (`restakeValidator`), increasing the dPOL backing.
2. Direct donations or transfers affect the balances.

Because there is no slippage parameter, a user's transaction is vulnerable to:

- Sandwich Attacks: An attacker can manipulate the exchange rate (e.g., by triggering a large reward claim or donation) immediately before the user's transaction to give them a worse rate, and then reverse the action or profit from the shift.
- Volatile Execution: Even without malice, if a large reward compound transaction lands before a user's buy, the user will receive fewer sPOL tokens than they estimated when signing the transaction.

This issue is present in both the sPOLController (L1) and sPOLChild (L2) contracts.

Recommendations:

Add a minOut parameter to all exchange functions (buySPOL, sellSPOL, etc.).

Customer Response:

Acknowledged.

M-10: Incorrect unit comparison in _sellSPOLSingle allows bypassing validator withdrawal limits

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/sPOLController.sol#L511	Status: Fixed	

Description:

The _sellSPOLSingle function allows users to withdraw their stake from a specific validator. To maintain the protocol's target stake distribution, it calls _maxRedeem(validator) to calculate the maximum amount of POL (dPOL) that can be withdrawn from that validator without violating the maxDivergence parameter.

The vulnerability is that _maxRedeem returns a value denominated in POL/dPOL (the underlying asset), but the function compares it directly against _amount, which is the input amount in sPOL.

```
uint256 maxRedeem = _maxRedeem(validator); // Returns POL/dPOL amount
require(_amount <= maxRedeem, ...);        // _amount is sPOL
```

Since the exchange rate between sPOL and POL is dynamic (and typically 1 sPOL >= 1 POL as rewards accrue), comparing them directly is incorrect. If 1 sPOL is worth 2 POL, a user can input 100 sPOL (worth 200 POL) and pass the check if maxRedeem is 100. This results in the withdrawal of 200 POL, violating the intended limit by 100%.

Recommendations:

The check must be performed using the same units. The input _amount (sPOL) should be converted to its POL/dPOL equivalent before being compared to maxRedeem.

Customer Response:

Fixed in PR#4.

Fix Review:

Confirmed.

M-11: _withdrawPOL incorrectly deletes withdrawal nonce 0, leading to fund loss

Severity: Medium	Impact: High	Likelihood: Low
Files: src/sPOLController.sol#L586	Status: Fixed	

Description:

The _withdrawPOL(address _user) function processes a user's pending withdrawals by iterating through their userNonces array. It uses a memory array memNonces to track status and a cleanup loop to compact the storage array.

In the first loop, if a withdrawal is processed, its slot in memNonces is set to 0 to mark it for removal:

```
if (shares == 0) { continue; }
totalAmount += shares;
// ...
memNonces[i] = 0; // Mark as processed
```

In the second loop (the cleanup phase), the code checks if memNonces[i] == 0 to decide whether to remove the item from storage:

```
if (memNonces[i] == 0) {
    userNonces[_user].pop();
} else {
    userNonces[_user][foundNonces] = memNonces[i];
    foundNonces++;
}
```

The flaw is that 0 is a valid nonce ID. The globalWithdrawNonce starts at 0. If a user has a pending withdrawal with nonce 0 that is not yet ready (shares == 0), the first loop will continue, leaving memNonces[i] as 0 (its original valid value).

However, the second loop interprets this 0 as "processed/empty". It skips adding it to the compacted list (else block) and instead performs a pop(). This effectively deletes the valid

pending withdrawal nonce 0 from the user's storage. The user permanently loses their claim to these funds.

The user who initiates the very first withdrawal in the protocol (nonce 0) will have their withdrawal request deleted if they (or anyone) call withdrawPOL before the withdrawal is ready.

Recommendations:

globalWithdrawNonce should start from 1 instead of 0.

Customer Response:

Fixed in PR#4.

Fix Review:

Confirmed

M-12: Front-running of buySPOLPermit allows redirection of stake to arbitrary validators

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/sPOLController.sol#L370	Status: Acknowledged	

Description:

The buySPOLPermit function allows a user to authorize a POL transfer via an ERC-2612 signature and simultaneously stake those tokens with a specific validator. The function signature is:

```
function buySPOLPermit(uint256 _amount, uint16 _validator, ...)
```

The issue is that the permit signature (r, s, v) only authorizes the sPOLController contract to spend _amount of the user's POL. It does not encode or commit to the _validator argument.

An attacker (such as a competing validator operator) can observe a pending buySPOLPermit transaction in the mempool. They can extract the valid signature and submit their own transaction calling buySPOLPermit with:

1. The same signature and user address.
2. The same `_amount`.
3. A different `_validator` ID (e.g., their own validator).

The sPOLController will successfully execute the permit, transfer the user's POL, and stake it with the attacker's chosen validator. The user's original transaction will then revert because the permit nonce has been consumed.

Recommendations:

The user should sign a message that includes both the permit details and the specific _validator ID they wish to fund. The buySPOLPermit function would then verify this comprehensive signature before executing.

Customer Response:

Acknowledged. No impact, users get what they want, selecting validators is not a feature, but an optimization.

M-13: _applyPermit uses incorrect token contract, bricking buySPOLWithDPOLPermit

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/sPOLController.sol#L872	Status: Fixed	

Description:

The _applyPermit function is a generic internal helper intended to handle permit execution for various tokens. It accepts an address _token argument to specify which token contract to call.

However, the implementation contains a hardcoded error in the else block:

```
if (address(_token) == address(sPOLToken)) {
    sPOLToken.consumePermit(...);
} else {
    // BUG: Hardcoded to use polToken instead of the passed _token
    polToken.permit(_user, address(this), _amount, _deadline, _v, _r, _s);
}
```

This logic incorrectly assumes that if the token is not sPOL, it must be polToken.

This assumption fails for the buySPOLWithDPOLPermit function, which passes the address of a Validator Share contract (dPOL) as the _token argument. The buySPOLWithDPOLPermit function is completely non-functional. Users cannot use permits to swap Validator Shares (dPOL) for sPOL

Recommendations:

Update _applyPermit to use the _token argument for the permit call in the else block, ensuring it interacts with the correct contract.

Customer Response:

Fixed in PR#4.

Fix Review:

Confirmed.

M-14: Underflow in _migrateValidator caused by restaked amount discrepancy

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/sPOLController.sol#L268	Status: Fixed	

Description:

The sPOLController contract manages validator delegations, including migrating stakes between validators. The migrateValidator(uint16 _oldValidator, uint16 _newValidator) function (without an amount parameter) calculates the total amount to migrate by adding the current totalStaked and the pending liquidRewards:

```
uint256 amount = validators[_oldValidator].totalStaked;
amount +=
validators[_oldValidator].validatorContract.getLiquidRewards(address(this));
_migrateValidator(_oldValidator, _newValidator, amount, true);
```

Inside _migrateValidator, if _restake is true, it calls restakeValidator(_oldValidator). This function claims rewards and restakes them.

The issue arises because the actual amount restaked is not guaranteed to equal the liquidRewards fetched earlier.

The ValidatorShare contract's restakePOL (which is what happens during restaking) involves purchasing shares (_buyShares). The amountRestaked (shares converted back to tokens) can be less than the input liquidReward:

```
        if (liquidReward != 0) {
            amountRestaked = _buyShares(liquidReward, 0, user);

            if (liquidReward > amountRestaked) {
                // return change to the user
                _payout(liquidReward - amountRestaked, user, "Insufficent
rewards", pol);
                stakingLogger.logDelegatorClaimRewards(validatorId, user,
liquidReward - amountRestaked);
```

```
            }

            (uint256 totalStaked,) = getTotalStake(user);
            stakingLogger.logDelegatorRestaked(validatorId, user, totalStaked);
        }
```

However, _migrateValidator unconditionally subtracts the *calculated* _amount (Total + Rewards) from the totalStaked after the restake:

```
if (validators[_oldValidator].status != ValidatorStatus.INACTIVE) {
    // _amount was calculated as (OriginalTotal + Rewards)
    // totalStaked was updated to (OriginalTotal + ActualRestakedAmount)
    validators[_oldValidator].totalStaked -= _amount;
}
```

If ActualRestakedAmount < Rewards (which might be common), then _amount > totalStaked. This subtraction causes an integer underflow, reverting the transaction.

Recommendations:

The migration logic should rely on the actual post-restake balance rather than a pre-calculated prediction.

Customer Response:

Fixed in an architectural redesign.

Fix Review:

Confirmed.

M-15: Front-running of permit signatures causes griefing in sPOLController

Severity: Medium	Impact: Low	Likelihood: High
Files: src/sPOLController.sol#L872	Status: Acknowledged	

Description:

The sPOLController contract provides several convenience functions that integrate permit functionality to allow users to approve and execute actions in a single transaction. These include:

- `buySPOLPermit`
- `buySPOLWithDPOLPermit`
- `sellSPOLPermit`

These functions call the internal _applyPermit helper:

```
function _applyPermit(...) internal {
    // ...
    if (address(_token) == address(sPOLToken)) {
        sPOLToken.consumePermit(...);
    } else {
        polToken.permit(...);
    }
    require(token.nonces(_user) == nonceBefore + 1, InvalidPermit());
}
```

The issue arises because the permit signature (r, s, v) is visible in the mempool. An attacker can extract this signature and submit it directly to the token contract (e.g., polToken.permit(...)) before the user's transaction is executed. This would grief the user's transaction as the permit was already consumed.

Recommendations:

Use a try/catch in case the permit was already consumed. **Note:** Using try/catch would allow allow any user to buy sPOL on behalf of another user as long as that user has given sufficient approval to the sPOLController.

Customer Response:

Acknowledged. The trade-off is worse.

M-16: DPOL and POL are used interchangeably which will lead to multiple issues

Severity: Medium	Impact: High	Likelihood: Low
Files: src/sPOLController.sol	Status: Acknowledged	

Description:

There are multiple instances throughout the sPOLController in which DPOL and POL are used interchangeably.

Let's take a simple example of a user buying sPOL with POL:

```
uint256 toMint = convertPOLtoSPOL(_amount);
        ValidatorInfo storage validator = validators[_validator];
        require(validator.status == ValidatorStatus.ACTIVE,
ValidatorNotActive(_validator));
        uint256 maxAmount = _validatorMaxTotalStakeDistance(validator, true);
        require(_amount <= maxAmount, ValidatorOverfunded(_amount, maxAmount));
        _takePOL(_amount, _user);

        uint256 actualShares = _buySharesFromValidator(validator, _amount);
        require(actualShares == _amount, BuySharesMismatch(_amount,
actualShares));
        lastSuccessfulBuyValidator = _validator;
        _mintSPOL(_user, _amount, toMint);
```

The POL amount is converted to SPOL, and the "actualShares" which are the DPOL amount, come from within the validator share contract:

```
(uint256 amountDeposited, uint256 liquidReward) =
_validator.validatorContract.restakeAndStakePOL(_amount);

//restakeAndStakePOL
uint256 amountToDeposit = _buyShares(amountPlusReward, amountPlusReward,
msg.sender);
        require(amountToDeposit == amountPlusReward, "exchange rate not 1");
```

```
        // transferring POL from sender, total amountToDeposit - liquidReward
        require(stakeManager.delegationDepositPOL(validatorId, _amount,
msg.sender), "deposit failed");

        return (amountToDeposit, liquidReward);

//And now _buyShares:

 function _buyShares(
        uint256 _amount,
        uint256 _minSharesToMint,
        address user
    ) private onlyWhenUnlocked returns (uint256) {
        require(delegation, "Delegation is disabled");

        uint256 rate = exchangeRate();
        uint256 precision = _getRatePrecision();
        uint256 shares = _amount.mul(precision).div(rate);
        require(shares >= _minSharesToMint, "Too much slippage");
        require(unbonds[user].shares == 0, "Ongoing exit");

        _mint(user, shares);

        // clamp amount of tokens in case resulted shares requires less tokens
than anticipated
        _amount = rate.mul(shares).div(precision);

        stakeManager.updateValidatorState(validatorId, int256(_amount));
        activeAmount = activeAmount.add(_amount);

        return _amount;
    }
```

It's expected that the amount of shares might differ from the amount of tokens, and this could be due to numerous factors, but let's take something as inevitable as precision loss, when tokens are "clamped":

```
// clamp amount of tokens in case resulted shares requires less tokens than
anticipated
_amount = rate.mul(shares).div(precision);
```

Example:

- exchangeRate: 250_000e18 * 1e18 / 250_100e18 = 0.996e18
- shares: 5000e18 * 1e18 / 0.996e18 = 5020e18
- So now _amount will be transformed to:
  - _amount = 0.996e18 * 5020e18 / 1e18 = 4999.9e18

First Instance: DoS and Frequent Failures

Within buy functions we have this hard requirement, which could frequently fail due to said difference in exchange rate, or just precision loss. This requirement dictates that the amount returned from the ValidatorShare contract needs to be equal to the one inputted, but this not always is the truth, even due to something as small as precision loss, as mentioned above.

```
require(amountDeposited == _amount, IncorrectValidatorShareExchangeRate(_amount,
amountDeposited));
```

Second Instance

The amount returned to the user is the "clamped" amount, but on the other hand the shares minted to the sPOLController, aren't the same number, so:

```
uint256 shares = _amount.mul(precision).div(rate);
        require(shares >= _minSharesToMint, "Too much slippage");
        require(unbonds[user].shares == 0, "Ongoing exit");

        _mint(user, shares);

        // clamp amount of tokens in case resulted shares requires less tokens
than anticipated
```

```
        _amount = rate.mul(shares).div(precision);

        stakeManager.updateValidatorState(validatorId, int256(_amount));
        activeAmount = activeAmount.add(_amount);

        return _amount;
```

To use the above example, the sPOL Controller will get minted 5020e18 shares, while it will return _amount, which will then be used within the sPOL Controller to increase the totalStaked and subsequently the totalDPOL

```
_adddPOLBalance(amountDeposited);
validators[_validator.index].totalStaked += amountDeposited + liquidReward;
```

So the balanceOf the sPOLController will be 5020e18, but the totalStaked will be 5000e18, leading to a discrepancy between the actual balance of the contract, and the virtual amount which is recorded.

Third Instance

When buying SPOL with DPOL, the migrations flow will be invoked in order to transfer the amount from the old validator to the new validator:

```
function _migrateValidator(uint16 _oldValidator, uint16 _newValidator, uint256
_amount, bool _restake) internal {
        if (_restake) {
            restakeValidator(_oldValidator);
            restakeValidator(_newValidator);
        }
        stakeManager.migrateDelegation(_oldValidator, _newValidator, _amount);
        if (validators[_oldValidator].status != ValidatorStatus.INACTIVE) {
            validators[_oldValidator].totalStaked -= _amount;
        }
        validators[_newValidator].totalStaked += _amount;
        emit ValidatorMigrated(_oldValidator, _newValidator, _amount);
    }
```

- `migrateDelegation` calls `migrateOut`:

```
function migrateOut(address user, uint256 amount) external onlyOwner {
       _withdrawAndTransferReward(user, true);
       (uint256 totalStaked, uint256 rate) = getTotalStake(user);
       require(totalStaked >= amount, "Migrating too much");

       uint256 precision = _getRatePrecision();
       uint256 shares = amount.mul(precision).div(rate);
       _burn(user, shares);
```

The amount passed in the migrateOut is assumed to be POL, not DPOL, that's why shares are calculated, but this isn't the case here, as we're calculating DPOL shares on a DPOL amount, rather than a POL one.

This might lead to multiple impacts:

- Contract/validator has less shares due to the different in exchangeRate;
- The amount returned isn't clamped as with other cases which again might affect the exchange ratio;

Everywhere else, we're calculating DPOL shares based on a POL amount, but here we're calculating DPOL shares based on a DPOL amount.

This is also present in migrateIn in which DPOL amount is "mistaken" for POL and shares are calculated.

Recommendations:

- Strict requirement needs to be removed, and rather replaced with a looser check which makes sure that the amount returned is within a certain max deviation.
- The amount which affects the virtual variables should be the same one which is returned from the ValidatorShare.

Customer Response:

Acknowledged. dPOL:POL ratio will always be 1:1.

M-17: lastSuccessfulBuyValidator and lastSuccessfulSellValidator are mistaken to be array indexes while they are validatorIds

Severity: Medium	Impact: Low	Likelihood: High
Files: src/sPOLController.sol	Status: Fixed	

Description:

There are multiple instances throughout the sPOLController in which lastSuccessfulBuyValidator/lastSuccessfulSellValidator are mistaken to be array indexes, while in reality they are validator IDs.

lastSuccessfulBuyValidator's main purpose is to intelligently distribute the stake to the next validator in line, or remove stake from someone else besides the last validator.

Let's take buySPOLSingle as an example:

```
lastSuccessfulBuyValidator = _validator;
```

lastSuccessfulBuyValidator is set as the validatorID, for example ID 56, the same is also true for the multi-buy/sell functions.

The first instance where IDs are mistaken for indexes is in _removeFromActiveValidators which is invoked after a validator is frozen or removed:

```
for (uint256 i = 0; i < activeValidators.length; i++) {
            if (activeValidators[i] == _validator) {
                activeValidators[i] = activeValidators[activeValidators.length -
1];
                activeValidators.pop();
                if (lastSuccessfulBuyValidator >= i) {
                    lastSuccessfulBuyValidator = 0;
                }
                if (lastSuccessfulSellValidator >= i) {
                    lastSuccessfulSellValidator = 0;
                }
```

```
                break;
```

We're checking if lastSuccessfulBuyValidator (which is a validatorID), whether it's greater than an index. If we have 5 active validators, indexes would be from 0 to 4, but the IDs could be 75,88,92,32,15 for example.

The second instance of this case is in _selectValidators:

```
for (uint256 i = 0; i < activeValidators.length; i++) {
            ValidatorInfo storage validator =
                validators[activeValidators[(lastSuccessfulBuyValidator + i) %
activeValidators.length]];
```

- We're performing a modulo operation between `lastSuccessfulBuyValidator` which is an ID (75,88...) with the length of activeValidators, which for example is 5.
- So if the last buy validator was ID 75 (index 0), in the first iteration we would have 75 + 0 % 5 = 0, and we will pick the validator with index 0, which is again 75, and start it from them, even though they were the `lastSuccessfulBuyValidator` . This could bring short term validator inconsistencies in their stakes, by picking the `lastSuccessfulBuyValidator` multiple times in a row, without continuing to validator index 1, which would be ID 88, as an example.

Recommendations:

Comparison needs to occur between two equal units, i.e. the lastSuccessfulBuy/SellValidator needs to be set as the index of the validator, instead of the ID.

Customer Response:

Fixed in PR#4.

Fix Review:

Confirmed. The lastSuccessfulBuyValidator and lastSuccessfulSellValidator functionality was removed.

Low Severity Issues

L-01: restakeAllActiveValidators reverts entirely if even a single validator is locked

Severity: Low	Impact: Low	Likelihood: Low
Files: src/sPOLController.sol#L269	Status: Acknowledged	

Description:

The restakeAllActiveValidators function is designed to iterate through the entire activeValidators array and compound rewards by calling restakePOL() on each validator's ValidatorShare contract.

However, in the Polygon ValidatorShare contract, the restakePOL() function internally calls _restake(), which subsequently calls _buyShares(). The _buyShares() function is protected by the onlyWhenUnlocked modifier:

```
// ValidatorShare.sol
function _buyShares(uint256 _amount, uint256 _minSharesToMint, address user)
private onlyWhenUnlocked returns (uint256) {
    // ...
}
```

If a validator becomes locked, any call to restakePOL() will revert. Because restakeAllActiveValidators executes these calls within a single continuous loop without error handling, a single locked validator will cause the entire transaction to revert.

Recommendations:

In restakeAllActiveValidators implement the same approach as in _selectValidators to skip validators who are locked.

Customer Response:

Acknowledged - The only way a validator becomes locked is when they unstake their shares, which doesn't occur on normal operation because once they unstake all there is no way to stake

again with the same validator address. In case the function call fails because of that we can just remove the validator from sPOL-controller and try again. We will implement this on our backend service.

L-02: Unchecked feeReceiver in initialization can permanently brick fee collection

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/sPOLController.sol#L147	Status: Fixed	

Description:

The initialize function accepts an _feeReceiver argument but does not check if it is address(0).

If feeReceiver is set to address(0):

1. Fees accrue normally in `feedPOLBalance`.
2. However, when `takeFee()` is called, it attempts `sPOLToken.mint(feeReceiver, ...)`. It will revert when minting to the zero address.
3. The `changeFeeReceiver` function calls `takeFee()` before updating the receiver address.

This means you cannot collect fees because the receiver is zero, and you cannot change the receiver because the update function tries to collect fees first.

Recommendations:

Add a non-zero check for _feeReceiver in the initialize function.

Customer Response:

Fixed in PR#4.

Fix Review:

Confirmed.

L-03: Nested restricted modifier on takeFee breaks role separation in changeFeeReceiver

Severity: Low	Impact: Low	Likelihood: Low
Files: src/sPOLController.sol#L780	Status: Fixed	

Description:

The sPOLController contract uses OpenZeppelin's AccessManagedUpgradeable to restrict access to sensitive functions. The changeFeeReceiver function is marked as restricted and internally calls takeFee to collect outstanding fees before updating the receiver address.

However, takeFee is a public function that is also marked as restricted.

In AccessManagedUpgradeable there is a comment saying that the restricted modifier should not be used on functions that are called inside the contract - like public and internal functions, but in our contract - sPOLController we have the takeFee function which is public and is called inside changeFeeReceiver which on it's own has the restricted modifier.

For example if we have a specific caller to take the fees say Alice that is specifically designed to call takeFee and another address Bob who is designed to call changeFeeReceiver. Then the changeFeeReceiver function can never be called because Bob is not allowed to call the takeFee and any attempt to change the fee recipient will lead to revert.

Recommendations:

Following OpenZeppelin's best practices, the restricted modifier should generally be avoided on functions called internally.

Customer Response:

Fixed in PR#4.

Fix Review:

Confirmed.

L-04: addValidator function performs no validations to make sure that the validator being added isn't an already existing one

Severity: Low	Impact: Low	Likelihood: Low
Files: src/sPOLController.sol	Status: Fixed	

Description:

The addValidator function fails to check whether the validator-in-question isn't already a part of the validators mapping, as well as active validators. It never checks whether that ID/validator contract is present, what their status is, totalStaked, etc.

This could lead to an already existing validator being re-introduced to the system which will zero-out their totalStaked, as well as their depositShare.

```
function addValidator(uint16 _validatorID) external restricted {
        require(stakeManager.isValidator(_validatorID),
ValidatorNotActive(_validatorID));

        IValidatorShare validatorContract =
IValidatorShare(stakeManager.getValidatorContract(_validatorID));
        require(address(validatorContract) != address(0),
ValidatorNotDelegating(_validatorID));

        validators[_validatorID] = ValidatorInfo({
            status: ValidatorStatus.ACTIVE,
            index: _validatorID,
            totalStaked: 0,
            validatorContract: validatorContract,
            depositShare: 0
        });
        validatorList.push(_validatorID);
        activeValidators.push(_validatorID);

        emit ValidatorAdded(_validatorID);
    }
```

Recommendations:

Introduce checks within the function which make sure that the validator being added isn't already present as part of the current roster of sPOLController validators.

Customer Response:

Fixed in PR#4.

Fix Review:

Confirmed.

Informational Severity Issues

I-01: _buySPOLWithDPOLMulti restakes after determining capacity which might lead to the overfunding of certain validators

Description:

In the else branch of the _buySPOLWithDPOLMulti the validator capacity is determined via _selectValidators prior to performing the restake via _restakeValidator. If during the _restakeValidator call there are any rewards accrued, this could result in the max amount of the validator being less than the one which was cached when _selectValidators was called.

```
 else {
            (uint16[] memory selectedValidators, uint256[] memory
selectedAmounts) = _selectValidators(_amount, true);

            // here we use transferFrom, without restake as we don't want to
change state of inactive validators
            bool success =
IValidatorShare(stakeManager.getValidatorContract(_validatorOfDPOL))
                .transferFrom(_user, address(this), _amount);
            require(success, DPOLRestakeTransferFromFailed());

            for (uint256 i = 0; i < selectedAmounts.length; i++) {
                if (selectedAmounts[i] == 0) {
                    continue;
                }
                _restakeValidator(selectedValidators[i]);
                if (validators[selectedValidators[i]].index != _validatorOfDPOL)
{
                    stakeManager.migrateDelegation(_validatorOfDPOL,
selectedValidators[i], selectedAmounts[i]);
                }
                validators[selectedValidators[i]].totalStaked +=
selectedAmounts[i];
            }
```

This could lead to a slight overfunding of certain validators.

Customer Response:

Acknowledged.

I-02: _sellVoucher_new should have internal visibility instead of public

Files:

contracts/staking/validatorShare/ValidatorShare.sol#L406

Description:

In the ValidatorShare contract, the _sellVoucher_new function acts as a shared helper for the user-facing sellVoucher_new and sellVoucher_newPOL functions, differentiating their behavior via a boolean flag (bool pol).

Following standard Solidity naming conventions - and the conventions used throughout the rest of this contract (e.g., _sellVoucher is private, _unstakeClaimTokens is internal, _buyShares is private) - functions prefixed with an underscore (_) should not be exposed to the public ABI. However, _sellVoucher_new is currently declared as public.

Recommendations:

Change the visibility of the _sellVoucher_new function from public to internal.

Customer Response:

Acknowledged.

I-03: Insufficient precision for validator deposit shares limits stake distribution granularity

Files:

src/sPOLController.sol#L653-L658

Description:

The sPOLController manages validator target shares using uint8 values representing whole percentages (0-100).

The protocol cannot allocate fractional percentages (e.g., 2.5% to a smaller validator). This lack of granularity may be problematic as the validator set grows or when trying to optimize yields across many validators with small capacity differences.

Recommendations:

Increase the precision of the share calculation. Instead of using 100 as the denominator (1%), use Basis Points (BPS) where 100% = 10,000.

Customer Response:

Acknowledged. We don't need more granularity for now.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.